
GPUNTOBIT
Il libro di *Triad*
are il linguaggio e il codice del progetto triad2.0,
primi programmi al seed in codice macchina
enti ┬À II Triad-Core ┬À III Triad-Full ┬À IV Dentro il progetto
fattoriale(n) = n < 2 ? 1 : n * fattoriale(n - 1)
edizione ┬À Standard del linguaggio 0.2.0 ┬À 2026
github.com/GPuntobit/triad2.0

Prefazione

Triad è un linguaggio di programmazione autonomo: sintassi a blocchi e tipi dichiarati, multiple dispatch, programmazione logica integrata, codice come dato. La sua definizione è una specifica normativa indipendente dall'implementazione, accompagnata da una specifica eseguibile e da una suite di conformità con golden verificabili. Questo libro ne è il percorso di studio.

Il libro nasce da un'esigenza semplice: imparare Triad non significa solo leggere la specifica, ma costruire un modello mentale del linguaggio, dei suoi costrutti e del suo runtime. La specifica è normativa e precisa, ma è scritta per essere implementata, non per essere letta come un romanzo. Questo volume la traduce in un percorso: spiegazioni, esempi, esercizi, marcatori di profilo.

Il lettore ideale sa già programmare in un altro linguaggio: sa cosa sono una variabile, una funzione, un ciclo, una ricorsione. Questo libro non insegna la programmazione in generale, insegna Triad. Le influenze del linguaggio (Julia, Prolog, Lisp) sono documentate nell'Appendice A della specifica: sono note informative, non definizioni. Triad non è un dialetto di nessuno dei tre.

Il volume è organizzato in quattro parti. La prima, *Fondamenti*, introduce il lessico, i valori e i tipi, il controllo di flusso, le funzioni, il dispatch, la logica, il codice come dato, gli errori, i contratti, il budget, le tracce e gli effetti. La seconda, *Triad-Core*, copre il sottoinsieme eseguibile oggi dall'interprete di riferimento, con tutti i 37 casi di conformance. La terza, *Triad-Full*, presenta i costrutti dello standard completo, oggi studiabili ma non ancora eseguibili dal runtime corrente. La quarta, *Dentro il progetto*, accompagna il lettore nel codice del repository: il seed in codice macchina, la suite di conformance, la CLI, gli strumenti per gli editor.

Due profili di conformità attraversano tutto il libro. Triad-Core è il sottoinsieme di bootstrap: valori, funzioni, chiusure, controllo di flusso, liste, stringhe, I/O. Triad-Full è lo standard completo. Ogni programma Core è anche un programma Full valido e deve comportarsi allo stesso modo. I marcatori **[Core]** e **[Full]** indicano in quale profilo esiste ciascun costrutto; i blocchi di codice sono marcati `triad` (eseguibile oggi) o `triad-full` (specificato ma non ancora eseguibile).

Il runtime di riferimento è il seed S0: un interprete Triad-Core scritto direttamente in codice macchina x86-64, senza alcun linguaggio ospite. Il piano di bootstrap prevede S1 (un interprete Triad scritto in Triad-Core), S2 (compilatore nativo), S3 (self-hosting) e S4 (parità cross-platform). Il libro riflette questo stato: dove il runtime non copre un costrutto, il testo lo dice esplicitamente.

Buona lettura, e buon codice.

— GPuntobit

Manuale di Triad 2.0

> *Il manuale completo del linguaggio: lessico, valori, controllo di flusso, funzioni, dispatch, logica, codice come dato, errori, contratti, budget, tracce, effetti, esercizi e riferimento.*

Triad è un linguaggio di programmazione autonomo: sintassi a blocchi e tipi dichiarati, multiple dispatch, programmazione logica integrata e codice come dato, in una sola sintassi. La sua definizione è una specifica normativa indipendente da qualsiasi implementazione (`SPEC.md`, standard `0.2.0`), accompagnata da una specifica eseguibile (`SPEC-CORE.md`) e da una suite di conformità con golden verificabili (`conformance/`).

Questo manuale insegna il linguaggio per come è definito dallo standard. Non è un riassunto della specifica: è un percorso di studio, con spiegazioni, esempi ed esercizi.

Come leggere questo manuale

A chi è indirizzato

Sai programmare, in qualunque linguaggio: sai cosa sono una variabile, una funzione, un ciclo, una ricorsione. Questo manuale non spiega la programmazione in generale, spiega **Triad**.

Due profili, una sola lingua

Lo standard definisce due profili di conformità, ed è utile saperli distinguere prima di cominciare:

profilo	che cosa contiene	dove è definito
Triad-Core	valori, funzioni, chiusure, controllo di flusso, liste, stringhe, I/O	`SPEC-CORE.md`
Triad-Full	tutto: tipi dichiarati, dispatch, logica, macro, contratti, budget, tracce, effetti	`SPEC.md`

Triad-Core è un **sottoinsieme** di Triad-Full: ogni programma Core è anche un programma Full valido e deve comportarsi allo stesso modo. Il manuale presenta i due profili **sullo stesso piano**, per costruzione logica: ogni capitolo introduce un costrutto e dice in quale profilo esiste. Non perché Full sia "più avanti", ma perché sono due facce della stessa specifica.

Dove la differenza conta, trovi due marcatori:

- **[Core]** — il costrutto esiste nel sottoinsieme di bootstrap;
- **[Full]** — il costrutto esiste solo nello standard completo.

Convenzioni di notazione

notazione	significa
`monospace`	codice Triad o un valore letterale
Output:	l'output esatto prodotto dal programma
[Core] / **[Full]**	il profilo in cui il costrutto esiste
`SPEC §n`	la sezione normativa corrispondente

I blocchi di codice sono marcati con il loro livello di eseguibilità:

fence	significa
```` ```triad ````	programma **eseguibile oggi** con l'interprete di riferimento
```` ```triad-full ````	programma **specifica dello standard**: definito e normativo, ma non ancora eseguibile dal runtime corrente

Come eseguire gli esempi

Gli esempi marcati `triad` si eseguono con l'interprete di riferimento JavaScript:

```
node seed/dev/triad_core.cjs programma.triad
```

Il programma riceve gli argomenti in `ARGS` (lista di stringhe, senza il nome del programma) e lo shebang `#!` in prima riga viene ignorato dal parser.

```
#!/usr/bin/env triad
# saluto.triad
println("ciao, " * "Triad")
println(6 * 7)
```

```
node seed/dev/triad_core.cjs saluto.triad uno due
```

Output:

```
ciao, Triad
42
```

Parte I — Il linguaggio

1. Lessico

1.1 Identificatori

Un identificatore comincia con una lettera o `_`, seguito da lettere, cifre o `_`. Un `!` finale è ammesso se non è seguito da `=`. **[Core]**

```
x = 1
somma_2 = 2
_ = 3
nome_valido = 4
```

`push!`, `sort!` e simili sono nomi legittimi: la convenzione del `!` finale è per le funzioni che mutano il loro argomento (cap. 9).

In Triad-Full gli identificatori ammettono anche lettere non ASCII. **[Full]**

```
età = 42
```

1.2 Numeri

Gli interi si scrivono con cifre decimali; `_` funziona come separatore. Il segno è un operatore unario, non parte del letterale. **[Core]**

```
println(42)
println(1_000_000)
println(-5)
```

Output:

```
42
1000000
-5
```

`Float` esiste solo in Triad-Full. **[Full]**

```
println(3.14)
println(1e-9)
```

1.3 Stringhe

Una stringa è una sequenza di byte UTF-8 **immutabile**, tra apici doppi. Gli escape sono `\n`, `\t`, `\"`, `\\`. **[Core]**

```
println("ciao\n")
println("tab:\t|")
println("virgolette: \"")
println("backslash: \\")
```

Output:

```
ciao

tab:    |
virgolette: "
backslash: \
```

In Core **non esiste l'interpolazione**: `"x = $x"` stampa letteralmente `x = $x`. In Full esiste, e si scrive `$x` per un'espressione semplice e `$(espr)` per una complessa. **[Full]**

```
x = 42
println("x = $x")
println("x + 1 = $(x + 1)")
```

1.4 Simboli

Un simbolo è `:` seguito da un nome o da un operatore: `:alice`, `:+`. Due simboli sono uguali se hanno lo stesso nome. In Triad-Full il simbolo è anche l'atomo della logica (cap. 16). **[Core]**

```
println(:alice)
println(:+)
println(:ok == Symbol("ok"))
```

Output:

```
alice
+
true
```

1.5 Commenti

`#` commenta fino a fine riga. `#= … =#` commenta su più righe; in Triad-Full i commenti annidati sono ammessi. **[Core]**

```
#= commento
   su più righe =#
x = 1_000   # commento di riga
println(x)
```

Output:

```
1000
```

1.6 Separatori e fine-riga

Un'istruzione termina con un a capo o con `;`. Gli a capo **chiudono** un'istruzione, tranne dentro `( )`, `[ ]` e `:( )`, e tranne subito dopo un operatore binario o una virgola di `rule`.

```
println(1); println(2)
```

Una conseguenza pratica: **in Triad-Core ogni a capo chiude l'istruzione**. La grammatica di Core non ha una regola di continuazione di riga: un'espressione spezzata su più righe non è un programma valido. Scrivi **una istruzione per riga** e usa `;` per separare due istruzioni.

```
totale = 1 + 2 + 3
println(totale);
println("seconda istruzione")
```

Output:

```
6
seconda istruzione
```

> ***In Triad-Full** la regola è più permissiva: `SPEC §1` stabilisce che un a capo **non** chiude l'istruzione dentro `( )`, `[ ]` e `:( )`, né subito dopo un operatore binario. L'interprete di riferimento attuale non implementa ancora questa eccezione, quindi anche in Full la forma sicura resta una istruzione per riga.*

1.7 La regola dei due punti e del punto interrogativo

`:` ha tre significati e `?` ne ha due. La specifica li chiama esplicitamente «il problema classico di `:`» — è il primo punto in cui Triad si discosta dalla sintassi che conosci.

scrittura	significato
`1:n`, `xs[2:end]`	`:` attaccato a un valore e senza spazi → **range**
`f(:a)`, `x = :a`	`:` non preceduto da un valore e seguito da un nome → **simbolo**
`:( … )`	→ **codice quotato**
`c ? a : b`	`:` circondato da spazi → **separatore del ternario**

scrittura	significato
`?x`	`?` attaccato → **variabile logica**
`?`	`?` con spazio dopo → **ternario**

Da qui la regola pratica: quando `:` è ambiguo, **metti o togli gli spazi**. `xs[2:end]` è un range; `c ? a : b` è un ternario perché i due spazi lo circondano.

1.8 Parole chiave

Lo standard definisce **29 parole chiave**:

```
function struct macro end if elseif else for in while let begin ret
break continue fact rule solve quote try catch with requires ensure
true false nothing isa is
```

E **4 parole contestuali**, valide solo dopo `solve`:

```
limit depth steps trace
```

`true`, `false` e `nothing` sono valori, non parole strutturali.

---

2. Valori e tipi

2.1 I tipi

tipo	valori	`typeof(v)`
`Int`	interi con segno a 64 bit, aritmetica con wrap	`Int`
`Float`	numeri in virgola mobile **[Full]**	`Float`
`Bool`	`true`, `false`	`Bool`
`String`	sequenza di byte UTF-8 immutabile	`String`
`Symbol`	`:nome`	`Symbol`
`List`	lista eterogenea mutabile, indici da 1	`List`
`Function`	funzione nominata, lambda, builtin	`Function`
`Nothing`	`nothing`	`Nothing`
`Expr`	codice quotato **[Full]**	`Expr`
`Term`, `LVar`	termini e variabili logiche **[Full]**	—
`Solution`	`{who = :dave}` **[Full]**	—
`Err`	errore come dato **[Full]**	—
`Type`	i tipi stessi sono valori **[Full]**	—
struct utente	`Point(3.0, 4.0)` **[Full]**	`Point`

Il set dei tipi Core è **chiuso e piccolo**: `Int`, `Bool`, `String`, `Symbol`, `List`, `Function`, `Nothing`. In Core non esistono `Float`, `Term`, `Solution`, `Err` come valori del linguaggio: un errore è una lista taggata `[Err, kind, msg]` (cap. 11) e una struct è `[Point, x, y]`.

`typeof` restituisce il **nome del tipo come simbolo**.

```
xs = [1, :a, "s"]
println(typeof(1), " ", typeof("s"), " ", typeof(:a), " ", typeof(x
println(typeof(nothing), " ", typeof(true), " ", typeof(x -> x))
```

Output:

```
Int String Symbol List
Nothing Bool Function
```

2.2 Gerarchia dei tipi

`Int <: Number`, `Float <: Number`, e ogni tipo è sottotipo di `Any`. `Any` è il supertipo usato nelle firme quando un parametro accetta qualunque cosa. **[Full]**

```
function accetta_qualsiasi(x::Any)
    "qualunque cosa: " * string(x)
end
```

2.3 Struct

Una `struct` è un record immutabile con campi tipizzati e costruttore posizionale. Il costruttore controlla i campi; un `Int` passato a un campo `Float` viene convertito, un valore non convertibile produce `Err(:type)`. **[Full]**

```
struct Point
    x::Float
    y::Float
end

p = Point(3, 4)
println(p.x, " ", p.y)
println(typeof(p) == Point)
println(p isa Point)
```

Output (golden Full):

```
3.0 4.0
true
true
```

In Core lo stesso concetto si rappresenta con una **lista taggata**: `[Point, x, y]`, con il simbolo del tipo in prima posizione. È così che gli interpreti di `selfhost/` rappresentano le struct Full.

```
punto(x, y) = [:Point, x, y]
px(p) = p[2]
py(p) = p[3]
println(punto(1, 2), " ", px(punto(1, 2)))
```

Output:

```
[:Point, 1, 2] 1
```

2.4 Conversioni

`Int(x)`, `Float(x)`, `String(x)`, `Symbol(x)`. Il nome del tipo usato come funzione converte; il nome di una struct usato come funzione **costruisce**. **[Full]**

```
println(Int("42") + 1)
println(Float(3))
println(Symbol("ciao"))
println(String(42))
```

`Int(s)` e `Symbol(s)` esistono anche in Core, ma solo per stringhe e simboli. **[Core]**

```
println(Int("41") + 1)
println(Symbol("bob"))
```

Output:

```
42
bob
```

2.5 Asserzione di tipo

`e::T` restituisce `e` se è di tipo `T`, altrimenti produce `Err(:type)`. **[Full]**

```
x = 3
println(x::Int)
println(iserr(x::String))
```

2.6 Il vincolo del dichiarato

`x::T = e` — e un parametro `x::T` — **fissa** il tipo di quel nome nell'ambiente in cui nasce. Gli assegnamenti successivi sono controllati e convertiti contro `T`. Una nuova dichiarazione `x::U = …` sostituisce il vincolo; un nome non dichiarato con tipo resta libero; una rilegatura in un ambiente interno (`let x = 2.5`) non eredita il vincolo esterno. **[Full]**

```
x::Int = 5
x = 6            # bene
x = 5.5          # Err(:type)

y::Float = 1.0
y = 2            # 2.0: conversione Int -> Float

s = "a"
s = 3            # bene: s non è dichiarato con un tipo
```

2.7 Le condizioni accettano solo `Bool`

Non esiste il concetto di "truthiness". `if 1` è un errore, non `true`. Questo vale per `if`, `while`, `&&`, `||`, `!`, il ternario e i filtri delle comprehension. **[Core]**

```
println(1 < 2 && 2 < 3)
x = 3
println(x > 2 ? "grande" : "piccolo")
```

Output:

```
true
grande
```

2.8 Come stampa un valore

`print` e `println` concatenano gli argomenti **senza separatori**; `println` aggiunge un a capo.

valore	come viene stampato
`Int`	decimale
`Bool`	`true` / `false`
`String`	così com'è
`Nothing`	`nothing`
`Symbol`	**senza** `:` se passato direttamente, **con** `:` dentro una lista
`List`	`[a, b]` ; le stringhe dentro non hanno virgolette
`Function`	`#lambda(x)`

```
xs = [1, :a, "s", true, nothing, [2, 3]]
println(xs)
println(:alice)
print("senza ", "a capo")
println()
```

Output:

```
[1, :a, s, true, nothing, [2, 3]]
alice
senza a capo
```

L'uguaglianza confronta numeri, stringhe e simboli **per valore**, le liste **elemento per elemento** (anche annidate), le funzioni **per identità**. **[Core]**

```
println([1, [2, [3]]] == [1, [2, [3]]])
println("ab" == "a" * "b", " ", :a == :a, " ", 1 == 2)
```

Output:

```
true
true true false
```

---

3. Variabili e assegnamento

`nome = espr` lega o aggiorna un nome. L'assegnamento **è un'espressione** e vale il valore assegnato. `+=`, `-=`, `*=` aggiornano una variabile esistente e valutano il lato sinistro una volta sola. Usare un nome mai assegnato è un errore di esecuzione. **[Core]**

```
x = 42
y = x + 8
x = x + 1
z = (x + y) * 2
println(x, " ", y, " ", z)
n = 10
n += 5
n -= 3
n *= 2
println(n)
```

Output:

```
43 50 186
24
```

Non esistono dichiarazioni: le variabili sono dinamiche. Le regole di **visibilità** si vedono nel capitolo 8.

---

4. Operatori e precedenza

Il parser è un **Pratt** con dodici livelli. Dalla precedenza più bassa alla più alta:

livello	operatori	associatività
1	`=` `+=` `-=` `*=`	destra
2	`x -> e`, `(a, b) -> e`, `c ? a : b`	destra
3	`||`	sinistra, cortocircuito
4	`&&`	sinistra, cortocircuito
5	`==` `!=` `<` `<=` `>` `>=` `in` `isa` `is`	sinistra
6	`a:b`	range, senza spazi
7	`+` `-`	sinistra
8	`*` `/` `%`	sinistra
9	`-x` `!x` `+x` `$x`	prefissi
10	`^`	**destra**
11	`e::T`	asserzione di tipo
12	`f(args)` `xs[i]` `a.campo`	postfissi, senza spazio prima di `(` e `[`

I livelli 6, 8 (la divisione), 9 (`$x`), 11 (`::`) e alcuni operatori di confronto esistono solo in Triad-Full.

4.1 Aritmetica intera

- `+ - * ^` su `Int` con **wrap a 64 bit** (complemento a due);
- `%` è il **resto con il segno del dividendo**, come `rem` di Julia: `-7 % 3 == -1`;
- `^` con esponente negativo è un errore; `%` per zero è un errore;
- `/` **non esiste in Core**: la divisione è di Triad-Full.

```
println(2 + 3 * 4, " ", (2 + 3) * 4, " ", 2 ^ 3 ^ 2)
println(7 % 3, " ", -7 % 3, " ", 10 % 4)
println(9223372036854775807 + 1)
```

Output:

```
14 20 512
1 -1 2
-9223372036854775808
```

Nota `2 ^ 3 ^ 2 == 512`: l'esponente è associativo a destra, come in matematica.

4.2 Logica e confronti

`&&` e `||` cortocircuitano. I confronti **non** si concatenano: `1 < 2 < 3` è un errore (si confronta `Bool < Int`); scrivi `1 < 2 && 2 < 3`. **[Core]**

```
println(!false, " ", true && false, " ", false || true)
println(1 < 2 && 2 < 3)
```

Output:

```
true false true
true
```

---

5. Controllo di flusso

In Triad **tutto è un'espressione**: `if`, `while`, `let`, `begin` hanno un valore, quello dell'ultima istruzione eseguita (`nothing` se non ce n'è).

5.1 `if` / `elseif` / `else` [Core]

```
function classifica(x)
    if x < 0
        "negativo"
    elseif x == 0
        "zero"
    else
        "positivo"
    end
end
println(classifica(-3), " ", classifica(0), " ", classifica(7))
v = if 2 > 1 :si else :no end
println(v)
```

Output:

```
negativo zero positivo
si
```

Il ternario `c ? a : b` è la forma breve di `if` e ha la stessa precedenza del `->`. **[Core]**

5.2 `while`, `break`, `continue` [Core]

`break` e `continue` agiscono sul ciclo **più interno**.

```
i = 0
somma_dispari = 0
while i < 10
    i += 1
    if i % 2 == 0
        continue
    end
    somma_dispari += i
end
println(somma_dispari)
while true
    break
end
println(:fine)
```

Output:

```
25
fine
```

In Core **non esiste** `for`: si itera con un contatore e `while`. In Full `for` esiste (cap. 5.6).

5.3 `let` e `begin` [Core]

`let` introduce un nuovo scope con legami locali; `begin … end` raggruppa istruzioni **senza** creare legami nuovi.

```
x = 1
let x = 10
    let x = 20
        println(x)
    end
    println(x)
end
println(x)
r = begin
    a = 2
    a * 21
end
println(r)
```

Output:

```
20
10
1
42
```

5.4 `for` e i range [Full]

`for` itera su una lista, un range o un risultato di `solve`. `a:b` produce il range da `a` a `b` inclusi.

```
s = 0
for i in 1:10
    s += i
end
println(s)

xs = [1, 2, 3]
println(xs[end])
println(xs[2:end])
```

Output (golden Full):

```
55
3
[2, 3]
```

`end` dentro un indicatore è l'ultimo elemento. La destrutturazione nel `for` è trattata nel capitolo 24.

5.5 Le comprehension [Full]

La comprehension `[e for x in xs]` costruisce una lista; con `if` aggiunge un filtro.

```
println([x^2 for x in 1:4])
println([x^2 for x in 1:4 if x > 2])
```

Output (golden Full):

```
[1, 4, 9, 16]
[9, 16]
```

5.6 `try` / `catch` [Full]

In Full l'errore è un **valore**, quindi `try` ha due forme: con `catch` il ramo gestito produce il valore dell'espressione; senza `catch`, l'errore **diventa** il valore dell'espressione.

```
e = try div(1, 0) end
println(iserr(e), " ", e.kind)

r = try div(1, 0) catch err; 0 end
println(r)
```

Output (golden Full):

```
true domain
0
```

---

6. Funzioni

6.1 Tre forme di definizione [Core]

```
# forma breve: il corpo è un'espressione
quadrato(x) = x * x

# forma blocco: il valore è quello dell'ultima espressione
function calcola(n)
    m = n + 1
    m * 2
end

# funzione anonima assegnata a un nome
moltiplica = function (a, b)
    a * b
end

println(quadrato(4), " ", calcola(3), " ", moltiplica(6, 7))
```

Output:

```
16 8 42
```

6.2 `return`, ricorsione, arità

`return e` esce subito dalla funzione; `return` da solo restituisce `nothing`. Una funzione nominata è visibile nel proprio corpo, quindi la ricorsione diretta funziona. Il numero di argomenti deve coincidere con i parametri. **[Core]**

```
function taglia(n)
    if n > 100
        return :grande
    end
    :piccola
end
fattoriale(n) = n < 2 ? 1 : n * fattoriale(n - 1)
fib(n) = n < 2 ? n : fib(n - 1) + fib(n - 2)
mcd(a, b) = b == 0 ? a : mcd(b, a % b)
println(taglia(500), " ", taglia(5))
println(fattoriale(10), " ", fib(15), " ", mcd(48, 18))
```

Output:

```
grande piccola
3628800 610 6
```

6.3 Funzioni di ordine superiore

Le funzioni sono valori di prima classe: si passano, si restituiscono, si compongono. In Core `map`, `filter` e `reduce` **non esistono**: si scrivono a mano (cap. 22.1). In Full sono builtin. **[Core]**

```
function applica_tutti(f, xs)
    out = []
    i = 1
    while i <= length(xs)
        push!(out, f(xs[i]))
        i += 1
    end
    out
end
println(applica_tutti(x -> x * x, [1, 2, 3]))
```

Output:

```
[1, 4, 9]
```

In Full, `reduce(+, xs)` e `map(sq, xs)` funzionano perché anche gli **operatori sono funzioni**.

6.4 L'annotazione di effetto `!io`

Una funzione che stampa deve dichiarare `!io` nella firma. Core **accetta l'annotazione e la ignora**: scriverla rende il codice già pronto per Full. **[Core]**

```
function mostra() !io
    println("hello from block")
    20 + 22
end
println(mostra())
```

Output:

```
hello from block
42
```

---

7. Tipi e multiple dispatch

Una funzione generica è un insieme di metodi. Ridefinire la stessa firma sostituisce il metodo precedente. Tra i metodi applicabili (stessa arità, ogni argomento sottotipo del parametro) vince il **più specifico**. **[Full]**

```
area(n::Number) = :number
area(n::Int) = :int

println(area(1))
println(area(1.5))
```

Output (golden Full):

```
int
number
```

Un metodo A è più specifico di B se ogni suo parametro è sottotipo del corrispondente parametro di B e le firme differiscono.

- Se nessun metodo è applicabile: `Err(:method)`.
- Se più metodi sono massimali: `Err(:ambiguous)`.
- Se il nome è anche un builtin (per esempio `+` sui numeri), si usa il builtin.

Gli operatori si estendono come funzioni, ridefinendoli:

```
struct Point
    x::Float
    y::Float
end

function +(a::Point, b::Point)
    Point(a.x + b.x, a.y + b.y)
end

p = Point(1, 2) + Point(3, 4)
println(p.x, " ", p.y)
```

Output (golden Full):

```
4.0 6.0
```

7.1 Il tipo di ritorno

`::R` è controllato all'uscita, con la conversione `Int -> Float`. Se il valore non è conforme: `Err(:type)`. **[Full]**

```
function metà(n::Number)::Float
    n / 2
end
println(metà(3))
```

Output (golden Full):

```
1.5
```

7.2 Funzioni locali

Una `function` definita dentro un'altra funzione è una **chiusura locale**, senza dispatch. **[Full]**

```
function esterno(x)
    function interno(y)
        y * 2
    end
    interno(x) + 1
end
println(esterno(5))
```

---

8. Lambda, chiusure e scope

8.1 Lambda [Core]

```
doppio = x -> x * 2
somma = (a, b) -> a + b
applica = f -> f(3)
println(doppio(5), " ", somma(1, 2), " ", applica(x -> x * x))
```

Output:

```
10 3 9
```

Una lambda con corpo a più istruzioni si scrive con `begin … end` o con `function (…) … end`. Le lambda senza parametri (`() -> …`) **non** sono supportate: usa `function () … end`.

8.2 Chiusure

Una lambda cattura l'ambiente in cui è creata. Secondo `SPEC-CORE §5` le variabili catturate sono **condivise**: le mutazioni sono visibili. **[Core]**

```
function add(n)
    x -> x + n
end
f = add(10)
g = add(100)
println(f(5), " ", g(5))

function contatore()
    v = 0
    x -> begin
        v += x
        v
    end
end
acc = contatore()
acc(5)
println(acc(10))
```

Output:

```
15 105
15
```

> ***Attenzione.*** *Il seed Windows cattura* ***per valore*** *e copre solo l'ultimo legame del frame: un accumulatore come* `contatore` *può non funzionare sul seed. L'interprete JS e la specifica usano il modello condiviso.*

8.3 Le regole di scope

1. `function`, lambda, `let`, `while`, `for` e le comprehension introducono **ambienti**; un nome è visibile nell'ambiente in cui è legato e in quelli interni (scope lessicale), mai in quelli esterni.

2. **Dentro una funzione nominata** l'assegnamento lega nel frame locale: una funzione **non modifica i globali**.
3. **Dentro una lambda** l'assegnamento a un nome già legato fuori aggiorna quel legame.
4. **Al top level** i cicli possono aggiornare i globali.
5. `let x = …` crea un nuovo `x`: assegnare `x` dentro il `let` non tocca l'`x` esterno.

```
x = 1
function f()
    x = 2
    x
end
println(f(), " ", x)
c = 0
function incrementa()
    c += 1
end
incrementa()
println(c)
```

Output:

```
2 1
0
```

Per far uscire un risultato da una funzione, restituiscilo — oppure usa una lista, che è mutabile.

---

9. Liste

Una lista è eterogenea, annidabile e **mutabile**. Gli indici partono da **1**. **[Core]**

```
xs = [1, 2, 3]
println(xs[1], " ", length(xs))
push!(xs, 4)
xs[1] += 100
println(xs)
alias = xs
push!(alias, 5)
println(xs)
```

Output:

```
1 3
[101, 2, 3, 4]
[101, 2, 3, 4, 5]
```

`push!` **muta** la lista e la restituisce. Le liste sono riferimenti: due nomi possono indicare la stessa lista, come si vede sopra.

La somma degli elementi con un ciclo esplicito:

```
xs = [101, 2, 3, 4, 5]
tot = 0
i = 1
while i <= length(xs)
    tot += xs[i]
    i += 1
end
println(tot)
```

Output:

```
115
```

In Full esistono anche `reverse`, `sort`, `sum`, `first`, `last`, `rest`, `pop!`, `append!`, `vcat`, `collect` e l'operatore `x in xs`.

---

10. Stringhe, simboli e conversioni

operazione	significato
`a * b`	concatenazione di stringhe
`length(s)`	numero di **byte** UTF-8 (`length("é") == 2`)
`string(x)`	rappresentazione testuale di qualsiasi valore (`string(:a) == "a"`, `string([1, 2]) == "[1, 2]"`)
`Int(s)`	da stringa a intero (`Int("42") == 42`); da `Bool`: `0` / `1`
`Symbol(s)`	da stringa a simbolo (`Symbol("ciao") == :ciao`)
`typeof(x)`	tipo come simbolo

```
nome = "Triad"
saluto = "ciao " * nome
println(saluto, " (", length(saluto), " byte)")
println("risposta: " * string(40 + 2))
println(Int("41") + 1, " ", Symbol("bob"), " ", string(true))
```

Output:

```
ciao Triad (10 byte)
risposta: 42
42 bob true
```

Per accedere ai singoli byte di una stringa servono i primitivi di bootstrap (cap. 21).

---

11. Errori come dati

Questa è una delle idee portanti di Triad: **l'errore è un valore**, non un'eccezione che interrompe il programma.

11.1 La forma `Err`

Un errore è un valore `Err` con tre campi:

- `kind::Symbol` — la categoria, su cui si può fare dispatch;
- `msg::String` — il messaggio;
- `data` — un valore qualsiasi, spesso un record `Info(...)` con i dettagli.

[Full]

```
e = err(:mio, "ops", 42)
println(e.kind, " ", e.msg)
println(iserr(e))
```

11.2 Produrre e propagare

costrutto	effetto
`err(:kind, "msg"[, data])`	costruisce un `Err` **senza** sollevarlo
`throw(e)`, `error("msg")`	solleva (`error` produce `Err(:error, …)`)
`try b catch e h end`	se `b` solleva, `e` è legato all'`Err` e si valuta `h`
`try b end`	**l'errore diventa il valore** dell'espressione
`iserr(x)`	controlla se `x` è un `Err`

Tutti i fallimenti del linguaggio — divisione intera per zero, `sqrt(-1.0)`, indici fuori range, nessun metodo applicabile — arrivano come `Err` e **mai** come fallimenti fatali. **[Full]**

```
e = try div(1, 0) end
println(e.kind)
println(iserr(e))

r = try div(10, 2) catch err; 0 end
println(r)
```

Output (golden Full):

```
domain
true
5
```

11.3 Le categorie di errore

kind	quando
`:syntax`	errore di parse; `data` contiene `line`, `col` e `incomplete`
`:undef`	nome, funzione, macro o predicato non definito
`:type`	tipo sbagliato (campi, ritorno, `::`, condizioni non `Bool`)
`:method`	nessun metodo applicabile; `data.types` contiene i tipi degli argomenti
`:ambiguous`	più metodi ugualmente specifici
`:field`, `:bounds`, `:domain`	campo inesistente, indice fuori range, errore matematico
`:instantiation`	variabile logica non legata dove serve un valore
`:contract`	`requires` / `ensures` violata (cap. 12)
`:budget`	passi esauriti (cap. 13)
`:effect`	effetto non dichiarato (cap. 15)
`:host`	qualsiasi altro errore dell'interprete

Un errore non gestito al top level interrompe il programma: il messaggio riporta la riga dell'istruzione (`riga 12: …`) e il codice di uscita è `1`.

11.4 In Core: errori come liste taggate

Core non ha `try`/`catch` né il tipo `Err`: un errore **termina il processo** con exit code `1` e un messaggio diagnostico su stderr. Ma il pattern "errore come dato" si può già scrivere, ed è esattamente così che S1 implementerà Full. La convenzione è `[:Err, kind, msg]`. **[Core]**

```
err(kind, msg) = [:Err, kind, msg]
is_err(v) = typeof(v) == :List && length(v) == 3 && v[1] == :Err

function div_intera(a, b)
    if b == 0
        return err(:domain, "divisione per zero")
    end
    q = 0
    while a >= b
        a -= b
        q += 1
    end
    q
end

r = div_intera(7, 0)
if is_err(r)
    println("errore ", r[2], ": ", r[3])
end
println(div_intera(17, 5))
```

Output:

```
errore domain: divisione per zero
3
```

Nota che `&&` cortocircuita: `v[1]` viene valutato solo se `v` è una lista di 3 elementi.

---

12. Contratti

Un contratto è una precondizione (`requires`) o una postcondizione (`ensures`) dichiarata nel corpo di una funzione. **[Full]**

```
function safe_div(a::Number, b::Number)::Float
    requires b != 0
    ensures abs(result * b - a) < 1e-9
    a / b
end
```

- `requires c` viene controllata **nel punto in cui compare**: per convenzione all'inizio del corpo.
- `ensures c` viene registrata e controllata all'uscita della funzione, con `result` legato al valore restituito, **dopo** il controllo del tipo di ritorno.
- Una violazione produce `Err(:contract, "requires violata in safe_div: b != 0", Info(fn, clause, cond[, result]))`. `cond` contiene il **codice** della condizione: è un dato, si può ispezionare.
- Le condizioni sono **pure**: un effetto dentro un contratto dà `Err(:effect)`.
- Al top level `requires c` funziona come un'asserzione.
- `ensures` fuori da una `function` è un `Err(:syntax)`.

```
println(safe_div(6, 3))

e = try safe_div(1, 0) end
println(e.kind)
println(e.data.clause)
println(e.data.fn)
```

Output (golden Full):

```
2.0
contract
requires
safe_div
```

---

13. Budget

Un **budget** limita quanti passi il programma può consumare. Un passo è una chiamata, un'iterazione di ciclo o un passo di risoluzione. Quando il budget si esaurisce, **il valore del `with` è `Err(:budget)`**: il programma non si blocca e non viene interrotto. **[Full]**

```
r = with budget(steps = 10_000)
    while true end
end
println(r.kind)
```

Output (golden Full):

```
budget
```

I budget si annidano e vale il più stretto. Un `with` converte in valore **solo** l'esaurimento del proprio budget, mentre quello di un budget esterno continua a propagarsi. `solve … steps N` è il budget specifico di una query.

Il motore logico usa uno stack esplicito: la ricorsione infinita, anche sinistra, finisce in `Err(:budget)` e **non** in uno stack overflow.

---

14. Tracce

La traccia di un calcolo è un **valore**, non output stampato. **[Full]**

```
fib(n::Int) = n < 2 ? n : fib(n - 1) + fib(n - 2)

t = with trace
    fib(3)
end
println(t.result)
println(length(t.trace))
```

Output (golden Full):

```
2
10
```

`fib(3)` effettua 5 chiamate (`fib(3)`, `fib(2)`, `fib(1)`, `fib(0)`, `fib(1)`): la traccia registra **due eventi per ogni chiamata**, `:call` e `:exit`, quindi 10 eventi. Per `fib(5)` sono 15 chiamate e 30 eventi — è il valore atteso dal caso di conformance `garanzie_core`.

`with trace … end` restituisce `Traced(result, trace)` con gli eventi `:call` e `:exit` di ogni funzione generica chiamata nel corpo. `solve … trace` restituisce `Traced(result, trace)` dove `result` è la lista delle soluzioni e gli eventi sono le porte del modello a scatole di Byrd (`call`, `exit`, `redo`, `fail`) più `depth`, quando un ramo viene tagliato da `depth`.

Gli eventi sono record: si filtrano e si contano come ogni altro valore.

Glossario degli eventi di traccia (porte di Byrd)

Quando si usa `solve … trace`, ogni passo della ricerca produce un evento che registra **a che riferimento** (quale goal, quale clausola, quali variabili logiche) si sta lavorando. Le quattro porte del modello a scatole di Byrd sono:

porta	quando si verifica	significato
`:call`	si entra in un goal	inizio della valutazione della clausola candidata
`:exit`	unificazione **riuscita**	il goal è dimostrato: è il "success" della ricerca
`:redo`	backtracking	si torna indietro per cercare un'altra soluzione
`:fail`	nessuna clausola applicabile	il goal **fallisce**: nessuna soluzione per quel ramo

Note importanti:

- **"success" e "fail" non sono esiti generici**: sono le porte `:exit` e `:fail` della traccia, riferite al goal/clausola in corso. Il fallimento di un goal non è un'eccezione: si manifesta come **lista di soluzioni vuota** nel `result` di `Traced(result, trace)`.
- `depth` appare come evento aggiuntivo quando un ramo viene tagliato dal limite di profondità (`solve … depth N`), segnalando che la ricerca è stata interrotta prima di un `:exit` / `:fail`.
- Gli eventi sono **record ordinari**: si filtrano (`[e for e in t.trace if e.port == :fail]`), si contano (`length`), si ispezionano (`e.args`, `e.depth`) come ogni altro valore Triad.
- Il riferimento di ogni evento è il **goal corrente e le variabili logiche** (`?x`, `?y`, …) coinvolte in quel passo: leggendo la traccia si ricostruisce *quale* alternativa è stata provata e *perché* è fallita.

```
# Conta i fallimenti di una ricerca
t = solve path(:a, :c) trace
fails = [e for e in t.trace if e.port == :fail]
println(length(fails))
```

```
fib(n::Int) = n < 2 ? n : fib(n - 1) + fib(n - 2)
t = with trace
    fib(3)
end
println([e.args for e in t.trace if e.port == :call])
```

Output (golden Full):

```
[[3], [2], [1], [0], [1]]
```

---

15. Effetti dichiarati

Le funzioni sono **pure per default**. Gli effetti si dichiarano nella firma, dopo i parametri. **[Full]**

```
function saluta(nome::String) !io
    println("ciao, ", nome)
end

function impara(x) !state
    fact visto($x)
end
```

effetto	operazioni
`io`	`println`, `print`
`state`	`fact` / `rule` eseguiti dentro una funzione (modificano la knowledge base)

Il controllo è **dinamico**. Una funzione viene eseguita solo con gli effetti che dichiara, e solo se il chiamante li permette. Se una funzione pura chiama `println`, o chiama una funzione `!io`, si ottiene `Err(:effect)`; `data` contiene `fn`, `caller` e `missing`.

- Il top level permette tutti gli effetti.
- Le lambda ereditano gli effetti del contesto in cui vengono eseguite.
- Macro e contratti sono puri.
- Leggere la knowledge base (`solve`, chiamare un predicato) è puro.
- La mutazione locale di dati (`push!` su una lista, assegnamento a variabili locali) **non** è un effetto: modifica solo valori raggiungibili

dall'ambiente locale.

```
function rumorosa()
    println("faccio rumore")
end

function tranquilla()
    rumorosa()
end

e = try tranquilla() end
println(e.kind)
```

Output (golden Full):

```
effect
```

---

16. Logica

Triad integra la programmazione logica **nella sintassi a blocchi**: i fatti e le regole sono costrutti di primo livello, non un embedded language. **[Full]**

16.1 Fatti, regole, termini

```
fact parent(:tom, :bob)
fact parent(:bob, :jim)

rule ancestor(?a, ?d) :- parent(?a, ?d)
rule ancestor(?a, ?d) :- parent(?a, ?x), ancestor(?x, ?d)
```

Dentro un termine:

- `:atomo` è un simbolo; `?x` è una variabile logica; numeri, stringhe e liste sono se stessi;
- `f(t1, t2)` è un termine composto. Anche `?x + 1` è un termine (`+(?x, 1)`) e **non viene valutato**;
- un **nome nudo** è un valore dell'ambiente corrente, letto al momento in cui viene eseguito `fact`, `rule` o `solve`: `chi = :bob; solve parent(chi, ?c)`;
- `$(espr)` inserisce esplicitamente un valore calcolato: `fact eta(:bob, $(2026 - 1975))`;
- `[a, b]` e `[?h | ?t]` sono liste e pattern su liste;
- il nome di una struct con argomenti, come `Point(?x, ?y)`, unifica con le istanze di quella struct.

Le variabili di una clausola vengono **rinominate a ogni uso** (standardizzazione a parte). L'unificazione fa l'**occurs-check**: `solve ?x = f(?x)` non ha soluzioni.

16.2 I goal

goal	significato
`p(t…)`	predicato della knowledge base, risolto con SLD e backtracking, clausole in ordine
`g1 && g2`, `(g1, g2)`	congiunzione; nelle `rule` anche con `,`
`g1 || g2`	disgiunzione
`!g`, `not(g)`	negazione per fallimento
`not(b)` *fuori* da un goal	è una funzione: nega un `Bool` (`not(true) == false`)
`a = b`	unificazione
`?y is espr`	valuta `espr` (le variabili devono essere legate) e unifica con `?y`
`a < b`, `==`, `!=`, `<=`, `>`, `>=`	confronto tra valori, valutati
`f(t…)` con `f` funzione Triad	chiama la funzione con gli argomenti valutati; deve restituire `Bool`
`true`, `false`	successo, fallimento

Un predicato senza clausole con quell'arità, che non sia nemmeno una funzione, dà `Err(:undef, "predicato sconosciuto: p/2")`. Una variabile non legata dentro un'espressione dà `Err(:instantiation)`.

16.3 `solve` è un valore

`solve` **restituisce una lista** di `Solution`. Non stampa nulla: è un valore come un altro.

```
fact parent(:alice, :bob)
fact parent(:alice, :carol)
fact parent(:bob, :dave)

rule grandparent(?x, ?z) :- parent(?x, ?y), parent(?y, ?z)

sols = solve grandparent(:alice, ?who) limit 5
println(sols)
println(sols[1].who)
println(length(sols))
```

Output (golden Full):

```
[{who = :dave}]
dave
1
```

Ogni soluzione lega le variabili della query **non anonime**, nell'ordine in cui compaiono. Le variabili rimaste libere restano `?x#n`. `vars(sol)` restituisce i nomi legati.

Opzioni di `solve`:

- `limit n` — al massimo `n` soluzioni;
- `depth d` — i rami più profondi di `d` vengono tagliati (falliscono);
- `steps s` — al massimo `s` passi di risoluzione, altrimenti `Err(:budget)`; il default è `1 000 000`;
- `trace` — vedi cap. 14.

16.4 Un predicato usato come funzione

Chiamare un predicato dove ci si aspetta un valore restituisce un `Bool`: dice se il goal è dimostrabile.

```
if grandparent(:alice, :dave)
    println("alice è nonna di dave")
end
```

16.5 Esempi

Percorso con costo:

```
fact edge(:a, :b, 2)
fact edge(:b, :c, 3)
rule path(?x, ?y, ?c) :- edge(?x, ?y, ?c)
rule path(?x, ?y, ?c) :- edge(?x, ?z, ?c1), path(?z, ?y, ?c2), ?c i
println((solve path(:a, :c, ?costo))[1].costo)
```

Output (golden Full):

```
5
```

Append in stile Prolog:

```
rule app([], ?l, ?l)
rule app([?h | ?t], ?l, [?h | ?r]) :- app(?t, ?l, ?r)
println((solve app([1, 2], [3], ?r))[1].r)
println(length(solve app(?a, ?b, [1, 2, 3])))
```

Output (golden Full):

```
[1, 2, 3]
4
```

Negazione per fallimento:

```
fact parent(:tom, :bob)
persone = [:tom, :bob]
foglie = filter(p -> !parent(p, ?_), persone)
println(foglie)
```

Output (golden Full):

```
[:bob]
```

---

17. Codice come dato

Il parser produce **un solo tipo di nodo**, `Node(head, args)`. È lo stesso albero che il kernel esegue, che `:( … )` restituisce come valore e che `triad --tree` stampa come S-expression. **[Full]**

sintassi	nodo
`f(a, b)`, `a + b`	`(call f a b)`, `(call + a b)`
`a && b`, `a || b`	`(and a b)`, `(or a b)`
`x = e`	`(assign x e)`
`:alice`	`(sym alice)`
`a.x`, `xs[i]`	`(dot a x)`, `(index xs i)`
`[a, b]`, `[h | t]`	`(list a b)`, `(cons (list h) t)`
`[e for x in xs if c]`	`(comprehension e x xs c)`
`x -> e`	`(lambda (tuple x) e)`
`x::T`	`(typed x T)`
`"a $x"`	`(string "a " x)`
`if c a else b end`, `c ? a : b`	`(if c (block a) (block b))`
`while`, `for`, `let`, `begin`	`(while c body)`, `(for x iter body)`, `(let (tuple …) body)`, `(block …)`
`function f(a::T)::R !io … end`	`(function f (tuple (typed a T)) R (tuple io) body)`
`struct P; x::Float; end`	`(struct P (tuple (typed x Float)))`
`macro m(a) … end`, `@m(e)`	`(macro m (tuple a) body)`, `(macrocall m e)`
`:(e)`, `quote … end`, `$e`	`(quasi e)`, `(quasi (block …))`, `(unquote e)`
`fact t`, `rule h :- g1, g2`	`(fact t)`, `(rule h g1 g2)`
`solve g limit 5 trace`	`(solve g (tuple (assign limit 5) (assign trace true)))`

sintassi	nodo
`try b catch e h end`	`(try b e h)`
`with budget(steps = N), trace … end`	`(with (tuple (call budget (assign steps N)) trace) body)`

17.1 Quote, unquote, eval

```
c = :(a.x + f(b, 2))
println(c.head)
println(sexpr(c))

n = 7
quadrato = :($n * $n)
println(eval(quadrato))
```

Output (golden Full):

```
call
(call + (dot a x) (call f b 2))
49
```

Dentro un quote, `$x` e `$(e)` inseriscono il **valore** di un'espressione: se è codice viene innestato, se è un numero o una lista diventa una costante, se è un Symbol diventa un identificatore. `eval(c)` esegue codice **nello scope globale**. Gli altri builtin del gruppo "codice" sono `parse`, `unparse`, `sexpr`, `Expr` e `gensym`.

---

18. Macro

Una macro riceve gli argomenti **come codice**, non valutati, e restituisce codice, che viene poi eseguito al posto della chiamata `@nome(...)` nello scope del chiamante. **[Full]**

```
macro unless(c, body)
    :(if !$c; $body; end)
end

macro swap(a, b)
    t = gensym(:tmp)
    quote
        $t = $a
        $a = $b
        $b = $t
    end
end

x = 1
y = 2
@swap(x, y)
println(x, " ", y)
```

Output (golden Full):

```
2 1
```

L'espansione è **pura** (§15) e **non è igienica**: per i nomi temporanei si usa `gensym()`.

18.1 Codice che genera codice

```
function poly(coeffs)
    acc = :(0)
    i = 0
    for c in coeffs
        acc = :($acc + $c * x ^ $i)
        i += 1
    end
    acc
end

p = poly([1, 2, 3])
x = 2
println(eval(p))
```

Output (golden Full):

```
17
```

---

19. Builtin

ambito	funzioni
aritmetica e confronto	`+ - * / ^ % == != < <= > >= !`, `abs sqrt floor ceil round exp log sin cos div mod min max`; `*` concatena le stringhe
liste	`length isempty first last rest reverse sort sum push! pop! append! vcat collect`, `a:b`, `x in xs`
ordine superiore	`map filter reduce any all count`
stringhe e output	`string repr println print`
tipi	`typeof`, `isa`, `Int Float String Symbol`
errori	`err iserr throw error`
codice	`eval parse unparse sexpr Expr gensym`
logica	`vars(sol)`

Il sottoinsieme di Core è più piccolo: `+ - * % ^`, i confronti, `&& || !`, `length`, `push!`, l'indicizzazione `xs[i]`, `*` sulle stringhe, `Int`, `string`, `Symbol`, `print`, `println`, `typeof` (`SPEC-CORE §7`). In Core `map`, `filter`, `reduce`, `sum`, `sort`, `div` e `mod` **non esistono**: si scrivono a mano (cap. 22.1).

---

20. Riga di comando e REPL

L'interfaccia di `triad` è normativa (`SPEC §14`):

```
triad                           # REPL
triad FILE [ARG...]             # esegue FILE; ARGS == [ARG...]
triad -l FILE                   # carica FILE e apre il REPL
triad -i FILE                   # esegue FILE e poi apre il REPL
triad -e CODE                   # esegue CODE
triad -E CODE                   # esegue CODE e stampa il valore
triad --tree FILE               # stampa l'albero uniforme come S-
triad -q                        # REPL senza banner
triad -h, --help
triad -v, --version
triad -                         # legge lo script da stdin
```

- L'estensione `.triad` / `.td` si può omettere; lo shebang `#!` è ignorato.
- Il programma vede gli argomenti in `ARGS`, lista di stringhe, senza il nome del programma.
- Codici di uscita: `0` successo, `1` errore non gestito (su stderr), `2` uso non valido.

20.1 Il REPL

Il REPL è in stile Prolog:

```
triad> 1 + 2
3
triad> ?- ancestor(:tom, ?d)
?d = :bob ;
?d = :liz.
triad> parent(:jim, ?x)
false.
```

- I blocchi restano aperti finché non sono chiusi da `end`.
- `?- goal` (il `.` finale è facoltativo) esegue una query con le opzioni di `solve`: `limit`, `depth`, `steps`, `trace`.

- Un'istruzione che è un goal su un predicato della knowledge base è trattata come query.
- Dopo ogni soluzione: `;`, `n` o spazio per la successiva, `a` per tutte le rimanenti, invio o `.` per fermarsi. `false.` significa nessun'altra soluzione.
- Comandi: `:load FILE…` (`:l`), `:reload` (`:r`), `:listing [nome]` (`:ls`), `:tree CODICE`, `:reset`, `:help` (`:h`), `:quit` (`:q`, `halt`, Ctrl-D).
- Ctrl-C interrompe il calcolo in corso; un errore non gestito è stampato e il REPL continua.

---

21. Primitivi di bootstrap

Cinque primitive servono a S1 per leggere e analizzare il sorgente Triad. **Non fanno parte di Triad-Full** (`SPEC-CORE §8`): usale solo nel codice di bootstrap.

primitiva	semantica
`readfile(path)`	contenuto del file come `String`; errore se non leggibile
`strlen(s)`	numero di byte
`strbyte(s, i)`	byte (`Int` 0–255) alla posizione `i`, da 1
`strsub(s, a, b)`	sottostringa dai byte `a` a `b` inclusi, da 1
`exit(n)`	termina il processo con codice `n`

```
s = "abc 123 xyz"
cifre = 0
i = 1
while i <= strlen(s)
    b = strbyte(s, i)
    if b >= 48 && b <= 57
        cifre += 1
    end
    i += 1
end
println(cifre)
fine = 1
while fine <= strlen(s) && strbyte(s, fine) != 32
    fine += 1
end
println(strsub(s, 1, fine - 1))
```

Output:

```
3
abc
```

Un programma può leggere se stesso con `readfile(ARGS[1])`: è il primo passo di un interprete self-hosted.

---

22. Stile e pattern idiomatici

22.1 Funzioni di ordine superiore scritte in Core

In Core non esistono `map` / `filter` / `reduce`: si scrivono una volta, con `while` e lambda, e poi si usano ovunque.

```
function mappa(f, xs)
    out = []
    i = 1
    while i <= length(xs)
        push!(out, f(xs[i]))
        i += 1
    end
    out
end

function filtra(p, xs)
    out = []
    i = 1
    while i <= length(xs)
        if p(xs[i])
            push!(out, xs[i])
        end
        i += 1
    end
    out
end

function riduci(f, acc, xs)
    i = 1
    while i <= length(xs)
        acc = f(acc, xs[i])
        i += 1
    end
    acc
end

xs = [1, 2, 3, 4, 5, 6]
println(mappa(x -> x * x, xs))
```

```
println(filtra(x -> x % 2 == 0, xs))
println(riduci((a, b) -> a + b, 0, xs))
```

Output:

```
[1, 4, 9, 16, 25, 36]
[2, 4, 6]
21
```

22.2 Record come liste taggate

In Core non c'è `struct`: un record è una lista con un simbolo come tag in prima posizione.

```
punto(x, y) = [:Punto, x, y]
px(p) = p[2]
py(p) = p[3]
somma_punti(a, b) = punto(px(a) + px(b), py(a) + py(b))
p = somma_punti(punto(1, 2), punto(10, 20))
println(p, " ", p[1] == :Punto)
```

Output:

```
[:Punto, 11, 22] true
```

22.3 Ambiente come lista di associazioni

È la struttura usata dagli interpreti in `selfhost/`: una lista piatta `[nome1, valore1, nome2, valore2, …]`.

```
function env_get(env, nome)
    i = 1
    while i < length(env)
        if env[i] == nome
            return env[i + 1]
        end
        i += 2
    end
    nothing
end

function env_set(env, nome, valore)
    i = 1
    while i < length(env)
        if env[i] == nome
            env[i + 1] = valore
            return env
        end
        i += 2
    end
    push!(env, nome)
    push!(env, valore)
end

env = []
env_set(env, :x, 10)
env_set(env, :y, 20)
env_set(env, :x, 11)
println(env, " ", env_get(env, :x), " ", env_get(env, :z))
```

Output:

```
[:x, 11, :y, 20] 11 nothing
```

22.4 Stato mutabile condiviso

Poiché una funzione nominata non può riassegnare un globale, lo stato condiviso si tiene in una lista mutabile o in una chiusura.

```
stato = [0]
function tick()
    stato[1] += 1
end
tick()
tick()
println(stato[1])
```

Output:

```
2
```

22.5 Costruire stringhe

```
function ripeti(s, n)
    out = ""
    while n > 0
        out = out * s
        n -= 1
    end
    out
end

function unisci(xs, sep)
    out = ""
    i = 1
    while i <= length(xs)
        if i > 1
            out = out * sep
        end
        out = out * string(xs[i])
        i += 1
    end
    out
end

println(ripeti("ab", 3))
println(unisci([1, 2, 3], ", "))
```

Output:

```
ababab
1, 2, 3
```

22.6 Regole di stile

- Funzioni piccole con nomi chiari; caso base esplicito in ogni ricorsione; per profondità grandi preferisci `while`.
- Cicli con contatori espliciti (`i = 1 … while i <= length(xs)`).
- Non usare `fact`, `rule`, `solve`, `struct` e le altre parole chiave di Full come nomi: il codice deve restare valido anche in Triad-Full.
- Scrivi `!io` sulle funzioni che stampano: Core lo ignora, Full lo richiede.
- Metti le parentesi quando spezzi un'espressione su più righe.

Parte II — Esercizi

Ogni esercizio ha una soluzione verificata. Prova prima, poi apri il riquadro «Soluzione».

23. Esercizi guidati (Triad-Core)

Esercizio 1 — Potenza e somme

Senza usare `^`, scrivi `potenza(b, e)` ricorsiva e `somma_fino(n)` (somma `1 + … + n`) con `while`. Stampa `potenza(3, 13)`, `potenza(2, 10)` e `somma_fino(100)`.

▶ Soluzione

Esercizio 2 — Numeri primi

Stampa la lista dei numeri primi minori di 30.

▶ Soluzione

Esercizio 3 — Inverti una lista

Scrivi `inverti(xs)` che restituisca una nuova lista con gli elementi in ordine inverso.

▶ Soluzione

Esercizio 4 — Ordinamento per inserzione

Ordina in loco `[5, 2, 9, 1, 7]`.

▶ Soluzione

Esercizio 5 — Contatore di parole

Conta le parole (separate da spazi) di `"il linguaggio Triad "` usando `strbyte`.

▶ Soluzione

Esercizio 6 — Un mini-tokenizer

Trasforma `"12+x*3"` in una lista di token `[tipo, testo]`, con tipi `:num`, `:id`, `:op`. È il primo passo di un interprete.

▶ Soluzione

Esercizio 7 — Valutare un'espressione

Usando i token dell'esercizio 6, scrivi un valutatore a discesa ricorsiva per `+` e `*` con precedenza, dove le variabili vengono da una lista di associazioni. Suggerimento: tieni la posizione corrente in una lista `pos = [1]`, così le funzioni possono aggiornarla.

▶ Soluzione

Esercizio 8 — Un ambiente a dizionario

Estendi l'esercizio 5 con un ambiente che supporta anche la rimozione, e scrivi `ambiente_da_elenco` che costruisce un ambiente da coppie `[nome, valore]`. Verifica con `env_get` ed `env_set` del capitolo 22.3.

▶ Soluzione

24. Esercizi su Triad-Full

Questi esercizi usano i costrutti dello standard completo. Sono definiti dalla specifica e hanno output golden in `conformance/cases/`, ma **non sono ancora eseguibili** dal runtime corrente.

Esercizio 9 — Dispatch su struct

Definisci `struct Vec2` con due campi `Float` e tre metodi di `area`: uno per `Vec2`, uno per `Number` e uno per `Any`. Verifica che il metodo più specifico vinca.

▶ Soluzione

Esercizio 10 — Una base di conoscenza

Scrivi fatti e regole per `parent` e `sorella`, e usa `solve` per trovare le sorelle di `:bob` e i nipoti di `:alice`.

▶ Soluzione

Esercizio 11 — Contratti in pratica

Scrivi `dividi(a, b)` con `requires b != 0` e `ensures abs(result * b - a) < 1e-9`, e usa `try` per distinguere il caso di contratto violata dal caso di successo.

▶ Soluzione

Esercizio 12 — Una macro che misura

Scrivi la macro `@misura(label, espr)` che valuta l'espressione e stampa etichetta e valore, restituendo il valore.

▶ Soluzione

Parte III — Riferimento

25. Grammatica

25.1 Triad-Core (EBNF normativa)

```
program  = { sep } { stmt sep { sep } } ;
sep      = NEWLINE | ";" ;
stmt     = funcdef | "return" [ expr ] | "break" | "continue" | exp
funcdef  = "function" NAME "(" [ NAME { "," NAME } ] ")" block "end
         | NAME "(" [ NAME { "," NAME } ] ")" "=" expr ;
expr     = assegnamento ;
assegnamento = NAME ( "=" | "+=" | "-=" | "*=" ) expr | ternario ;
ternario = or_expr [ "?" expr ":" expr ] ;
or_expr  = and_expr { "||" and_expr } ;
and_expr = cmp_expr { "&&" cmp_expr } ;
cmp_expr = sum_expr { ( "==" | "!=" | "<" | "<=" | ">" | ">=" ) sum
sum_expr = mul_expr { ( "+" | "-" ) mul_expr } ;
mul_expr = pow_expr { ( "*" | "%" ) pow_expr } ;
pow_expr = unary { "^" unary } ;                    (* destra *)
unary    = ( "-" | "!" ) unary | postfix ;
postfix  = primary { "(" args ")" | "[" expr "]" } ;
primary  = INT | STRING | SYMBOL | "true" | "false" | "nothing" | N
         | "(" expr ")"
         | "[" [ expr { "," expr } ] "]"
         | "(" NAME { "," NAME } ")" "->" expr
         | "if" expr block { "elseif" expr block } [ "else" block ]
         | "while" expr block "end"
         | "let" [ NAME "=" expr { "," NAME "=" expr } ] block "end
         | "begin" block "end"
         | "function" "(" [ NAME { "," NAME } ] ")" block "end" ;
block    = { stmt sep { sep } } ;                   (* fino a end/e
```

25.2 Triad-Full (EBNF normativa)

```
program    = { sep } { stmt sep { sep } } ;
sep        = NEWLINE | ";" ;
block      = { sep } { stmt sep { sep } } ;

stmt       = funcdef | structdef | macrodef | fact | rule
           | "requires" expr | "ensures" expr
           | "return" [ expr ] | "break" | "continue"
           | "(" NAME { "," NAME } ")" "=" expr
           | expr ;

funcdef    = "function" fname "(" [ params ] ")" [ "::" TYPE ] [ "!
             block "end"
           | NAME "(" [ params ] ")" "=" expr ;
fname      = NAME | "+" | "-" | "*" | "/" | "%" | "^" | "==" | "<"
params     = param { "," param } ;
param      = NAME [ "::" TYPE ] | "(" NAME { "," NAME } ")" ;
structdef  = "struct" NAME sep { NAME [ "::" TYPE ] sep } "end" ;
macrodef   = "macro" NAME "(" [ NAME { "," NAME } ] ")" block "end"

fact       = "fact" term ;
rule       = "rule" term [ ":-" goal { "," goal } ] ;

expr       = (* Pratt: precedenze dalla più bassa alla più alta *)
  1  a = b   a += b   a -= b   a *= b                (destra)
  2  x -> e   (a, b) -> e   c ? a : b                (destra)
  3  ||
  4  &&
  5  ==  !=  <  <=  >  >=  in  isa  is
  6  a:b                                             (range, senz
  7  +  -
  8  *  /  %
  9  -x  !x  +x  $x                                  (unari)
 10  ^                                               (destra)
```

```
 11  e::T                                                (asserzione
 12  f(args)  xs[i]  a.campo                             (postfissi,

primary    = NUMBER | STRING | SYMBOL | LVAR | NAME | "true" | "fal
           | "(" expr ")" | "(" expr "," ... ")"       (* tupla: pa
           | "[" [ expr { "," expr } [ "|" expr ] ] "]"
           | "[" expr "for" NAME "in" expr [ "if" expr ] "]"
           | ":(" expr ")" | "quote" block "end"
           | "@" NAME "(" args ")"
           | "if" expr block { "elseif" expr block } [ "else" block
           | "while" expr block "end"
           | "for" NAME "in" expr block "end"
           | "for" "(" NAME { "," NAME } ")" "in" expr block "end"
           | "let" [ NAME "=" expr { "," NAME "=" expr } ] block "e
           | "begin" block "end"
           | "try" block [ "catch" [ NAME ] block ] "end"
           | "with" modifier { "," modifier } block "end"
           | "solve" goal { "limit" expr | "depth" expr | "steps" e
           | "function" "(" params ")" block "end" ;
modifier   = "budget" "(" "steps" "=" expr ")" | "trace" ;
```

---

26. Schede rapide

26.1 Valori

vuoi	scrivi
un intero	`42`, `1_000`
un float **[Full]**	`3.14`, `1e-9`
un booleano	`true`, `false`
il vuoto	`nothing`
una stringa	`"testo"`
un simbolo	`:alice`
una lista	`[1, 2, 3]`
una lambda	`x -> x + 1`
una funzione	`function (x) x end`
un record Core	`[:Punto, 1, 2]`
una struct **[Full]**	`Point(3, 4)`
codice come dato **[Full]**	`:(a + b)`, `quote … end`

26.2 Controllo di flusso

vuoi	scrivi
una condizione	`if c … elseif d … else … end`
una condizione breve	`c ? a : b`
un ciclo	`while c … end` **[Core]** · `for x in xs … end` **[Full]**
uscire dal ciclo	`break`
saltare al passo dopo	`continue`
uscire dalla funzione	`return e`
uno scope nuovo	`let x = 1 … end`
un blocco senza scope	`begin … end`
una lista costruita **[Full]**	`[x * x for x in xs]`
catturare un errore **[Full]**	`try b catch e h end`

26.3 Funzioni

vuoi	scrivi
una funzione	`f(x) = x + 1`
un corpo multi-istruzione	`function f(x) … end`
un tipo di ritorno **[Full]**	`function f(x)::Float … end`
un effetto **[Full]**	`function f(x) !io … end`
un predicato in un goal **[Full]**	`function evena(n) n % 2 == 0 end`
una precondizione **[Full]**	`requires cond`
una postcondizione **[Full]**	`ensures cond`

27. Tabella degli errori

27.1 In Core

Core non ha `try` / `catch` né valori `Err`: un errore **termina il processo** con exit code `1` e un messaggio diagnostico su stderr. Quello già stampato su stdout resta.

messaggio	causa
`parse error: unexpected token …` / `parse error: expected …`	sintassi non Core (`1.5` , `/` , `() -> …` , `end` mancante)
`error: undefined variable NOME`	nome mai assegnato, o funzione chiamata prima di essere definita
`error: not a function`	si chiama un valore che non è una funzione (`x = 5; x(1)`)
`error: arity mismatch`	numero di argomenti diverso dai parametri
`error: index out of range`	`xs[i]` con `i < 1` o `i > length(xs)`
`error: + type error` (e simili)	operandi di tipo sbagliato (`1 + "a"`)
`error: if cond` / `while cond` / `ternary cond` / `&& expects bool`	condizione non `Bool`
`error: div by zero` / `negative exponent`	`x % 0` , `2 ^ -1`
`error: unknown function NOME`	builtin riservato usato in modo non supportato
`cannot read file: …`	file inesistente o non leggibile
`usage: triad FILE [ARG...]` (exit 2)	nessun file passato

27.2 In Triad-Full

Gli stessi fallimenti arrivano come `Err` con un `kind` (cap. 11.3), invece di terminare il processo. Un errore non gestito al top level stampa `riga N: …` e esce

con codice `1`.

---

28. Glossario

termine	significato
Albero uniforme	l'unico tipo di nodo prodotto dal parser, `Node(head, args)`; è anche ciò che il kernel esegue
Ambiente	l'insieme dei legami di nomi a valori visibile in un punto del programma
Backtracking	il motore logico torna sui legami e prova la clausola successiva
Budget	limite di passi eseguibili; esaurito produce `Err(:budget)`
Clausola	un `fact` o una `rule` della knowledge base
Chiusura	funzione che cattura l'ambiente in cui è stata creata
Dispatch	scelta del metodo in base ai **tipi** degli argomenti
Effetto	capacità dichiarata di una funzione: `io`, `state`
Err	l'errore come dato: `kind`, `msg`, `data`
Goal	una domanda alla knowledge base: predicato, congiunzione, disgiunzione, unificazione
Knowledge base	l'insieme dei fatti e delle regole
Macro	una funzione che riceve e restituisce codice
Occurs-check	la regola che impedisce a `?x` di unificarsi con un termine che contiene `?x`
Precedenza	l'ordine in cui gli operatori vengono valutati (12 livelli)
Profilo	un sottoinsieme normativo dello standard: Core o Full
Scope lessicale	un nome è visibile dove è stato legato e all'interno, mai all'esterno
Soluzione	una `Solution`: i legami prodotti da un goal risolto
Standardizzazione a parte	le variabili di una clausola vengono rinominate a ogni uso
Unificazione	l'uguaglianza logica: due termini si rendono identici legando le variabili

termine	significato
Valori di prima classe	funzioni e operatori usabili come qualsiasi altro valore

---

29. Cosa gira oggi

Questa sezione non fa parte del linguaggio: è lo stato dell'implementazione,useful per orientarsi prima di scrivere codice.

profilo	stato
Triad-Core	**eseguibile**: interprete di riferimento JavaScript `seed/dev/triad_core.cjs` (37/37 casi Core) e, in parte, il seed Windows `seed/windows-x86_64/seed.exe`
Triad-Full	**non eseguibile**: struct, dispatch, logica, macro, contratti, budget, tracce, effetti, destrutturazione, `for`, `try`, `Float` sono specifica; nella conformance Full passa solo `errore`

Differenze note fra i due runtime Core:

funzionalità	interprete JS	seed Windows
`return`, `break`, `continue`	sì	non elencati nel README del seed
lambda	più parametri	**un solo parametro**
chiusure	condivise	**per valore**, solo l'ultimo legame del frame
`string(x)`	tutti i tipi	solo `Int` e `String`
liste	eterogenee, `xs[i]`, `length`, `push!`	solo interi; costruzione, stampa, `push!`
`get(xs, i)`	non esiste	helper **non standard**, indici **da 0**
`ARGS`, primitivi di bootstrap	sì	non elencati nel README del seed
messaggi di errore	su stderr	solo exit code

Regola pratica: **scrivi codice che rispetta `SPEC-CORE` e verificalo con l'interprete JS**; se deve girare anche sul seed, restringiti alle funzionalità elencate nella colonna di destra e verificalo su Windows.

Due limiti noti che il lettore deve conoscere prima di scrivere codice:

- **ogni a capo chiude l'istruzione**, anche dentro `( )`, `[ ]` e negli argomenti di una chiamata. L'eccezione prevista da `SPEC §1` per Triad-Full (a capo dentro le parentesi, o subito dopo un operatore binario) non è implementata nell'interprete di riferimento. Scrivere una istruzione per riga;
- **il seed Windows non implementa l'indicizzazione standard `xs[i]`**: lì c'è l'helper non standard `get(xs, i)`, con indici **da 0**.

Appendice A — Influenze dichiarate

Triad nasce da idee di Julia, Prolog e Lisp, ma **non è un dialetto di nessuno dei tre**: la definizione del linguaggio è la specifica.

da	idea ripresa
Julia	blocchi `function … end`, `struct`, tipi con `::`, multiple dispatch, operatori-funzione, `if` / `for` / `while` / `let`
Prolog	`fact` / `rule` come costrutti, unificazione con occurs-check, backtracking, `solve`, variabili logiche `?x`
Lisp	codice come dato, albero uniforme, `:( … )` / `quote` con `$`, macro

Propri di Triad, non presi in prestito: errori come dati, contratti, budget, tracce ed effetti dichiarati come parte del linguaggio, e l'integrazione dei tre modelli in una sintassi a blocchi.

Appendice B — Dove guardare

vuoi	leggi
la definizione normativa completa	`SPEC.md`
il sottoinsieme di bootstrap	`SPEC-CORE.md`
il percorso di studio per capitoli e il codice del progetto	`TUTORIAL.md`
gli esempi completi con output	`examples/`
il contratto eseguibile, con i golden	`conformance/`
lo stato del bootstrap	`STATUS.md` · `BOOTSTRAP.md`
il piano di lavoro	`plan.md` · `TASK.md`
gli strumenti per gli editor	`editors/`
la licenza	`LICENSE` — MIT

Manuale scritto per lo standard Triad `0.2.0`. Le sezioni normative sono in `SPEC.md` e `SPEC-CORE.md`; in caso di divergenza, la specifica prevail.



Triad 2.0 — Wiki: imparare il linguaggio e il codice del progetto

Questa wiki è il punto di ingresso per **imparare Triad** e per **imparare a scrivere il codice presente in questo repository**: programmi Triad, l'interprete di riferimento, il seed in codice macchina, l'implementazione self-hosted (S1), la suite di conformità, la CLI di sviluppo, il benchmark AI, i file per gli editor.

- **Standard:** `SPEC.md` (Triad-Full, versione in `VERSION`, `0.2.0`) e `SPEC-CORE.md` (Triad-Core, il sottoinsieme di bootstrap).
- **Contratto eseguibile:** `conformance/`.
- **Piano di bootstrap:** `BOOTSTRAP.md`; stato in `STATUS.md`; task in `TASK.md` e `plan.md`.
- **Estensioni dei sorgenti:** `.triad` (canonica) e `.td` (alias storico).

> ***Cosa gira oggi — leggere prima di iniziare.***

profilo	*dove è definito*	*eseguibile oggi?*
Triad-Core	`SPEC-CORE.md`	***sì**: interprete di riferimento JavaScript* `seed/dev/triad_core.cjs` *(37/37 casi Core) e, in parte, il seed Windows* `seed/windows-x86_64/seed.exe` *(33 casi Core verificati in CI)*
Triad-Full	`SPEC.md`	***no**: struct, dispatch, logica, macro, contratti, budget, tracce, effetti, destrutturazione,* `for`, `try`, `Float` *sono specifica; nella conformance Full passa solo* `errore`

L'implementazione Julia (`triad.jl`, `syntax.jl`, `test/` *) è stata rimossa: Triad non dipende da Julia. Gli output Triad-Full mostrati in questa wiki sono i **golden** prodotti a suo tempo da quella implementazione e congelati in* `conformance/cases/`.

*In questa wiki ogni esempio marcato **Output:** è un programma Triad-Core verificato con* `node seed/dev/triad_core.cjs`*; gli esempi Full sono marcati **Output (golden Full):** oppure non riportano output.*

Come usare questa wiki

Scegli un percorso:

vuoi…	leggi
scrivere programmi Triad che girano oggi	Parte I → Parte II (capitoli 4–17, con esercizi)
conoscere il linguaggio completo dello standard	Parte III (capitoli 18–32)
contribuire all'interprete JS, al seed, a S1, alla CLI o alla conformance	Parte IV (capitoli 33–43)
consultare grammatica, schede rapide, errori, glossario	Parte V (capitoli 44–49)

Convenzioni:

- I comandi sono mostrati per Linux/macOS (`sh`) e, quando diversi, per Windows (`cmd` /PowerShell).
- "Interprete JS" = `seed/dev/triad_core.cjs` , interprete di riferimento Triad-Core usato su Linux/macOS e dal runner di conformance. "Seed" = `seed/windows-x86_64/seed.exe` , interprete Triad-Core in codice macchina (S0). Dove si comportano diversamente lo dice il capitolo 15.

Parte I — Primi passi

1. Cos'è Triad

Triad è un linguaggio di programmazione autonomo che unisce tre paradigmi in una sintassi a blocchi:

influenza	idea ripresa in Triad
Julia	`function … end` , `struct` , tipi con `::` , multiple dispatch, operatori come funzioni, `if` / `for` / `while` / `let`
Prolog	`fact` , `rule` , unificazione con occurs-check, backtracking, `solve` con variabili logiche `?x`
Lisp	codice come dato (`Node`), `:( … )` / `quote` con `$` , macro

A questi si aggiungono, come parte del linguaggio e non di una libreria: **errori come dati** (`Err`), **contratti** (`requires` / `ensures`), **budget** (`with budget`), **tracce** (`with trace`) ed **effetti dichiarati** (`!io` , `!state`).

1.1 Due profili: Core e Full

Lo standard definisce due profili di conformità (SPEC §16.1):

- **Triad-Core** (`SPEC-CORE.md`): interi, Bool, stringhe, simboli, `nothing` , liste, funzioni, lambda e chiusure, `if` / `elseif` / `else` , `while` , `let` , `begin` , ternario, `return` / `break` / `continue` , più alcuni primitivi di bootstrap. È abbastanza piccolo da essere

implementato in codice macchina scritto a mano e abbastanza espressivo da scriverci l'interprete di tutto Triad.

- **Triad-Full** (`SPEC.md`): tutto il linguaggio.

Ogni programma Triad-Core (esclusi i primitivi di bootstrap) è anche un programma Triad-Full valido e deve comportarsi allo stesso modo.

1.2 La catena di bootstrap

Triad non vuole dipendere da nessun linguaggio ospite. Il piano (`BOOTSTRAP.md`):

```
S0  seed: interprete Triad-Core in codice macchina x86-64 (dump esa
S1  interprete di tutto Triad scritto in Triad-Core      (selfhost/
S2  compilatore nativo scritto in Triad                  (non inizi
S3  compilatore self-hosted: S2 compilato da S2          (non inizi
S4  Triad completo: REPL, CLI, tooling, parità Linux     (non inizi
```

Gli strumenti in Node.js (`seed/dev/*.js`, `conformance/run.mjs`, `cli/`) sono **strumenti di sviluppo**: aiutano a generare e verificare, ma non fanno parte della catena S0 → S4.

---

2. Installare ed eseguire

2.1 Requisiti

- **Node.js ≥ 18** (la CI usa Node 20) per l'interprete JS, il runner di conformance e la CLI.
- Su Windows, nient'altro per il seed: `seed.exe` si ottiene decodificando `seed.hex` con `certutil`.
- Git per clonare il repository: `git clone https://github.com/GPuntobit/triad2.0.git`.

2.2 Eseguire un file

piattaforma	comando
Linux / macOS	`node seed/dev/triad_core.cjs file.triad arg1 arg2`
Windows (seed)	`seed\windows-x86_64\seed.exe file.triad` oppure `bin\triad.cmd file.triad`
qualsiasi (stdin, interprete JS)	`echo 'println(7 * 6)' | node seed/dev/triad_core.cjs -`

Esempi pronti che girano oggi: tutti i `conformance/core/cases/*/program.td`.

```
node seed/dev/triad_core.cjs conformance/core/cases/ricorsione/prog
```

```
bin\triad.cmd conformance\core\cases\ricorsione\program.td
```

2.3 Materializzare il seed

Il sorgente del seed è il dump esadecimale; l'eseguibile si ottiene con un decodificatore di sistema:

```
Remove-Item seed\windows-x86_64\seed.exe -ErrorAction SilentlyConti
certutil -decodehex seed\windows-x86_64\seed.hex seed\windows-x86_6
```

```
# Linux: seed ELF allo stadio E0 (stampa solo "Triad seed Linux OK"
xxd -r -p seed/linux-x86_64/seed.hex seed/linux-x86_64/seed && chmo
```

2.4 Launcher `bin/triad` e `bin/triad.cmd`

Aggiungendo `bin/` al `PATH` si usa `triad file.triad`. I launcher cercano, in ordine:

1. `bin/triad-native` (eseguibile standalone, **non ancora prodotto**: arriverà con S2–S3);
2. `seed/<piattaforma>/seed` (su Windows `seed\windows-x86_64\seed.exe`).

Su Linux il launcher trova il seed ELF E0, che **non interpreta ancora programmi**: per eseguire codice su Linux usa direttamente l'interprete JS. `triad --build` stampa un messaggio ed esce con codice 2.

2.5 Codici di uscita

codice	significato
`0`	successo
`1`	errore di parsing o di esecuzione (messaggio su stderr), file non leggibile
`2`	uso non valido (es. nessun file)

Le opzioni di SPEC §14 (`-e`, `-E`, `-l`, `-i`, `--tree`, `--help`, REPL) **non sono ancora implementate**: il seed Windows esce con codice 0 senza output, l'interprete JS le tratta come nome di file.

2.6 La CLI di sviluppo in breve

```
bin/triad-cli status                # ambiente e seed trovato (Win
bin/triad-cli test conformance      # conformance Core
bin/triad-cli ai verify -v          # verifica le soluzioni di ai/
bin/triad-cli seed generate windows  # rigenera seed.hex e stampa l
node conformance/run.mjs            # runner di conformance diret
```

I launcher `bin/triad-cli*` eseguono `npm install` in `cli/` al primo avvio. In alternativa si usa l'eseguibile standalone `triad-cli.exe` / `triad-cli` allegato alla release beta. Dettagli nel capitolo 39.

---

3. Il primo programma

Crea `ciao.triad`:

```
#!/usr/bin/env triad
# il mio primo programma Triad
nome = "Triad"
println("ciao, " * nome)
println(6 * 7)
```

Output:

```
ciao, Triad
42
```

Eseguilo:

```
node seed/dev/triad_core.cjs ciao.triad          # Linux/macOS
```

```
seed\windows-x86_64\seed.exe ciao.triad          # Windows
```

Cosa notare:

- la prima riga `#!` (shebang) è ignorata: inizia con `#`, quindi è un commento;
- `println` stampa i suoi argomenti **senza separatori** e aggiunge un a capo;
- `*` tra stringhe è la **concatenazione** (non `+`);

- non c'è una funzione `main`: le istruzioni al top level si eseguono in ordine.

3.1 Argomenti da riga di comando

Gli argomenti dopo il nome del file sono in `ARGS`, una lista di stringhe (senza il nome del programma). Esempio con l'interprete JS:

```
printf 'println(ARGS)\nprintln(length(ARGS))\n' > args.triad
node seed/dev/triad_core.cjs args.triad uno due      # [uno, due]
```

> *Il seed Windows documenta solo l'uso `seed.exe FILE`: `ARGS` non è tra le funzionalità elencate in `seed/windows-x86_64/README.md`. Usa l'interprete JS per i programmi con argomenti.*

3.2 Il ciclo di lavoro consigliato

1. scrivi il programma usando solo Triad-Core (Parte II);
2. eseguilo con l'interprete JS (`node seed/dev/triad_core.cjs`);
3. se deve girare anche sul seed Windows, controlla la tabella del capitolo 15;
4. se è un comportamento del linguaggio da fissare, trasformalo in un caso di conformance (capitolo 38).

Parte II — Triad-Core passo per passo (eseguibile oggi)

Ogni capitolo di questa parte usa solo Triad-Core. Gli esempi con **Output:** sono stati eseguiti con l'interprete JS; il caso di conformance indicato in fondo a ogni capitolo è il riferimento normativo da leggere e far girare.

4. Lessico

elemento	forma	esempi
identificatore	lettera o `_`, poi lettere/cifre/`_`; un `!` finale è ammesso	`x`, `somma_2`, `push!`
intero	cifre decimali, `_` come separatore; il segno è l'operatore unario	`42`, `1_000_000`, `-5`
booleano	`true`, `false`	
nothing	`nothing`	
stringa	`"..."` con escape `\n \t \" \\`; **nessuna interpolazione**	`"ciao\n"`
simbolo	`:` seguito da un nome o da un operatore	`:alice`, `:+`
commento	`#` fino a fine riga; `#= ... =#` su più righe	
separatore	a capo oppure `;`	`println(1); println(2)`

Operatori di Core: `+ - * % ^ == != < <= > >= && || ! = += -= *= -> ?:`.

Parole riservate usate da Core: `function end if elseif else while let begin return break continue true false nothing`.

Non fanno parte di Core: letterali `Float` (`1.5`), variabili logiche `?x`, macro `@nome`, interpolazione `$x`, l'operatore `/`.

```
#= commento
   su più righe =#
x = 1_000   # commento di riga
println(x); println("tab:\t|", " virgolette: \"", " backslash: \\")
println(:alice, " ", :+)
```

Output:

```
1000
tab:     | virgolette: " backslash: \
alice +
```

> ***Attenzione.*** `"x = $x"` *in Core stampa letteralmente* `x = $x`*: l'interpolazione è solo Triad-Full. In Core si concatena:* `"x = " * string(x)`*.*
>
> *I commenti* `#= … =#` ***non si annidano*** *nell'interprete JS (SPEC Full richiede l'annidamento).*

Riferimento: `conformance/core/cases/aritmetica`, `simboli_base`, `stringhe_base`.

---

5. Valori, tipi e stampa

tipo	valori	`typeof(v)`
`Int`	interi con segno a 64 bit, aritmetica con wrap (complemento a due)	`Int`
`Bool`	`true`, `false`	`Bool`
`String`	sequenza immutabile di byte UTF-8	`String`
`Symbol`	`:nome`; uguali se hanno lo stesso nome	`Symbol`
`List`	lista eterogenea **mutabile**, indici da 1	`List`
`Function`	funzione nominata, lambda o chiusura	`Function`
`Nothing`	`nothing`	`Nothing`

`typeof` restituisce il **nome del tipo come simbolo**.

5.1 Regole di stampa di `print` / `println`

- concatenano gli argomenti senza separatori; `println` aggiunge `\n`;

- `Int` in decimale, `Bool` come `true` / `false`, `String` così com'è, `Nothing` come `nothing`;
- un `Symbol` passato direttamente è stampato **senza** `:`; dentro una lista **con** `:`;
- una `List` è stampata come `[a, b]`; le stringhe dentro una lista sono stampate senza virgolette.

```
xs = [1, :a, "s", true, nothing, [2, 3]]
println(xs)
println(:alice)
println(typeof(1), " ", typeof("s"), " ", typeof(:a), " ", typeof(x
println(typeof(nothing), " ", typeof(true), " ", typeof(x -> x))
print("senza ", "a capo")
println()
```

Output:

```
[1, :a, s, true, nothing, [2, 3]]
alice
Int String Symbol List
Nothing Bool Function
senza a capo
```

5.2 Uguaglianza

`==` e `!=` confrontano numeri, stringhe e simboli **per valore**, le liste **elemento per elemento** (anche annidate), le funzioni **per identità**.

```
println([1, [2, [3]]] == [1, [2, [3]]])
println("ab" == "a" * "b", " ", :a == :a, " ", 1 == 2)
```

Output:

```
true
true true false
```

Riferimento: `conformance/core/cases/valori`, `nothing_base`, `confronti`.

---

6. Variabili e assegnamento

- `nome = espr` lega o aggiorna un nome; l'assegnamento è un'espressione e vale il valore assegnato.
- `+=`, `-=`, `*=` aggiornano una variabile esistente (il lato sinistro è valutato una volta).
- Usare un nome mai assegnato è un errore di esecuzione (`undefined variable`).
- Non ci sono dichiarazioni né tipi: le variabili sono dinamiche.

```
x = 42
y = x + 8
x = x + 1
z = (x + y) * 2
println(x, " ", y, " ", z)
n = 10
n += 5
n -= 3
n *= 2
println(n)
```

Output:

```
43 50 186
24
```

Le regole di **visibilità** (dentro funzioni, `let`, lambda) sono nel capitolo 10.

Riferimento: `conformance/core/cases/variabili`.

---

7. Operatori ed espressioni

7.1 Precedenza (dalla più bassa alla più alta)

livello	operatori	associatività
1	`=` `+=` `-=` `*=`	destra
2	`c ? a : b`	destra
3	`||`	sinistra, cortocircuito
4	&&	sinistra, cortocircuito
5	`==` `!=` `<` `<=` `>` `>=`	sinistra
6	`+` `-`	sinistra
7	`*` `%`	sinistra
8	`^`	**destra**
9	unari `-` `!`	prefissi
10	`f(args)` `xs[i]`	postfissi

7.2 Aritmetica intera

- `+ - * ^` su `Int` con **wrap a 64 bit**;
- `%` è il **resto con il segno del dividendo** (come `rem` di Julia): `-7 % 3 == -1`;
- `^` con esponente negativo è un errore; `%` per zero è un errore;
- **non esiste** `/` in Core (la divisione è di Triad-Full).

```
println(2 + 3 * 4, " ", (2 + 3) * 4, " ", 2 ^ 3 ^ 2)
println(7 % 3, " ", -7 % 3, " ", 10 % 4)
println(9223372036854775807 + 1)
```

Output:

```
14 20 512
1 -1 2
-9223372036854775808
```

7.3 Logica e condizioni

- Le condizioni (`if` , `while` , ternario, `&&` , `||` , `!`) accettano **solo** `Bool` : non c'è "truthiness" (`if 1` è un errore).
- `&&` e `||` cortocircuitano.
- I confronti non si concatenano: `1 < 2 < 3` è un errore (si confronta `Bool < Int`); scrivi `1 < 2 && 2 < 3` .

```
println(!false, " ", true && false, " ", false || true)
println(1 < 2 && 2 < 3)
x = 3
println(x > 2 ? "grande" : "piccolo")
```

Output:

```
true false true
true
grande
```

Riferimento: `conformance/core/cases/aritmetica` , `operatori` , `interi` , `confronti` .

---

8. Controllo di flusso

In Triad **tutto è un'espressione**: `if`, `while`, `let`, `begin` hanno un valore (quello dell'ultima istruzione eseguita, `nothing` se nessuna).

8.1 `if` / `elseif` / `else`

```
function classifica(x)
    if x < 0
        "negativo"
    elseif x == 0
        "zero"
    else
        "positivo"
    end
end
println(classifica(-3), " ", classifica(0), " ", classifica(7))
v = if 2 > 1 :si else :no end
println(v)
```

Output:

```
negativo zero positivo
si
```

8.2 `while`, `break`, `continue`

```
i = 0
somma_dispari = 0
while i < 10
    i += 1
    if i % 2 == 0
        continue
    end
    somma_dispari += i
end
println(somma_dispari)
while true
    break
end
println(:fine)
```

Output:

```
25
fine
```

`break` e `continue` agiscono sul `while` più interno. Non esiste `for` in Core: si itera con un contatore e `while`.

8.3 `let` e `begin`

`let` introduce un nuovo scope con legami locali; `begin … end` raggruppa istruzioni senza creare legami nuovi.

```
x = 1
let x = 10
    let x = 20
        println(x)
    end
    println(x)
end
println(x)
r = begin
    a = 2
    a * 21
end
println(r)
```

Output:

```
20
10
1
42
```

Riferimento: `conformance/core/cases/controllo`, `if_else`, `if_elseif`, `if_nested`, `if_no_else`, `while_loop`, `let_annidato`, `begin_block`.

---

9. Funzioni

9.1 Tre modi di definire una funzione

```
# forma breve: il corpo è un'espressione
quadrato(x) = x * x

# forma blocco: il valore è quello dell'ultima espressione
function calcola(n)
    m = n + 1
    m * 2
end

# funzione anonima assegnata a un nome
moltiplica = function (a, b)
    a * b
end

println(quadrato(4), " ", calcola(3), " ", moltiplica(6, 7))
```

Output:

```
16 8 42
```

9.2 `return`, ricorsione, arità

- `return e` esce subito dalla funzione; `return` da solo restituisce `nothing`.
- Una funzione nominata è visibile nel proprio corpo: la ricorsione diretta funziona.
- Il numero di argomenti deve coincidere con i parametri (altrimenti `arity mismatch`).

```
function taglia(n)
    if n > 100
        return :grande
    end
    :piccola
end
fattoriale(n) = n < 2 ? 1 : n * fattoriale(n - 1)
fib(n) = n < 2 ? n : fib(n - 1) + fib(n - 2)
mcd(a, b) = b == 0 ? a : mcd(b, a % b)
println(taglia(500), " ", taglia(5))
println(fattoriale(10), " ", fib(15), " ", mcd(48, 18))
```

Output:

```
grande piccola
3628800 610 6
```

9.3 Annotazione di effetto `!io`

In Triad-Full una funzione che stampa deve dichiarare `!io` (capitolo 28). Core accetta l'annotazione e la ignora: scriverla rende il codice già pronto per Full.

```
function mostra() !io
    println("hello from block")
    20 + 22
end
println(mostra())
```

Output:

```
hello from block
42
```

Riferimento: `conformance/core/cases/funzioni`, `funzioni_semplici`, `funzioni_blocco`, `funzioni_io`, `ricorsione`, `gcd`, `formula`.

---

10. Lambda, chiusure e scope

10.1 Lambda

```
doppio = x -> x * 2
somma = (a, b) -> a + b
applica = f -> f(3)
println(doppio(5), " ", somma(1, 2), " ", applica(x -> x * x))
```

Output:

```
10 3 9
```

Una lambda con corpo a più istruzioni si scrive con `begin … end` o con `function (…) … end`. Le lambda senza parametri (`() -> …`) **non** sono supportate: usa `function () … end`.

10.2 Chiusure

Una lambda cattura l'ambiente in cui è creata. Secondo SPEC-CORE §5 le variabili catturate sono **condivise**: le mutazioni sono visibili.

```
function add(n)
    x -> x + n
end
f = add(10)
g = add(100)
println(f(5), " ", g(5))

function contatore()
    v = 0
    x -> begin
        v += x
        v
    end
end
acc = contatore()
acc(5)
println(acc(10))
```

Output:

```
15 105
15
```

> *Il seed Windows cattura **per valore** e copre solo l'ultimo legame del frame: un accumulatore come `contatore` può non funzionare sul seed (capitolo 15).*

10.3 Regole di scope

1. `function`, lambda, `let` e `while` introducono ambienti; un nome è visibile nell'ambiente in cui è legato e in quelli interni (scope lessicale).
2. **Dentro una funzione nominata** l'assegnamento lega nel frame locale: una funzione **non modifica i globali**.

3. **Dentro una lambda** l'assegnamento a un nome già legato fuori aggiorna quel legame (è così che funziona l'accumulatore sopra).
4. **Al top level** i cicli aggiornano i globali.
5. `let x = …` crea un nuovo `x`; assegnare `x` dentro il `let` non tocca l'`x` esterno.

```
x = 1
function f()
    x = 2
    x
end
println(f(), " ", x)
c = 0
function incrementa()
    c += 1
end
incrementa()
println(c)
```

Output:

```
2 1
0
```

Per far "uscire" un risultato da una funzione restituiscilo, oppure usa una lista (che è mutabile, capitolo 11).

Riferimento: `conformance/core/cases/lambda`, `closure`, `chiusure`, `chiusure_local`, `chiusura_in_if`, `let_chiusura`, `scope`.

---

11. Liste

- Letterale: `[1, 2, 3]`, `[]`, eterogenee e annidabili.
- Indici **da 1**: `xs[1]` è il primo elemento; fuori range è un errore.

- Assegnamento per indice: `xs[i] = v`, anche `xs[i] += v`.
- `length(xs)` numero di elementi; `push!(xs, v)` aggiunge in coda, **muta** la lista e la restituisce.
- Le liste sono riferimenti: due nomi possono indicare la stessa lista.

```
xs = [1, 2, 3]
println(xs[1], " ", length(xs))
push!(xs, 4)
xs[1] += 100
println(xs)
alias = xs
push!(alias, 5)
println(xs)
tot = 0
i = 1
while i <= length(xs)
    tot += xs[i]
    i += 1
end
println(tot)
```

Output:

```
1 3
[101, 2, 3, 4]
[101, 2, 3, 4, 5]
115
```

Core non ha `map`, `filter`, `reduce`, `sum`, `sort`: si scrivono in Core (vedi capitolo 16).

> *Sul seed Windows: liste solo di interi, `push!` sì, `xs[i]` e `length(xs)` **no**; c'è invece l'helper non standard `get(xs, i)` con indici **da 0** (capitolo 15).*

Riferimento: `conformance/core/cases/liste`, `liste_base`, `liste_push`.

---

12. Stringhe, simboli e conversioni

operazione	significato
`a * b`	concatenazione di stringhe
`length(s)`	numero di **byte** UTF-8 (`length("é") == 2`)
`string(x)`	rappresentazione testuale di qualsiasi valore (`string(:a) == "a"`, `string([1, 2]) == "[1, 2]"`)
`Int(s)`	da stringa a intero (`Int("42") == 42`); da Bool: `0` / `1`
`Symbol(s)`	da stringa a simbolo (`Symbol("ciao") == :ciao`)
`typeof(x)`	tipo come simbolo

```
nome = "Triad"
saluto = "ciao " * nome
println(saluto, " (", length(saluto), " byte)")
println("risposta: " * string(40 + 2))
println(Int("41") + 1, " ", Symbol("bob"), " ", string(true))
println(:ok == Symbol("ok"))
```

Output:

```
ciao Triad (10 byte)
risposta: 42
42 bob true
true
```

Per accedere ai singoli byte di una stringa servono i primitivi di bootstrap del capitolo successivo.

Riferimento: `conformance/core/cases/stringhe`, `stringhe_concat`, `stringhe_multi`, `simboli_base`.

---

13. Primitivi di bootstrap

Servono a S1 per leggere e analizzare sorgente Triad. **Non** fanno parte di Triad-Full (SPEC-CORE §8): usali solo nel codice di bootstrap.

primitiva	semantica
`readfile(path)`	contenuto del file come `String`; errore se non leggibile
`strlen(s)`	numero di byte
`strbyte(s, i)`	byte (`Int` 0–255) alla posizione `i`, da 1
`strsub(s, a, b)`	sottostringa dai byte `a` a `b` inclusi, da 1
`exit(n)`	termina il processo con codice `n`

Esempio: contare le cifre di una stringa e separare la prima parola.

```
s = "abc 123 xyz"
cifre = 0
i = 1
while i <= strlen(s)
    b = strbyte(s, i)
    if b >= 48 && b <= 57
        cifre += 1
    end
    i += 1
end
println(cifre)
fine = 1
while fine <= strlen(s) && strbyte(s, fine) != 32
    fine += 1
end
println(strsub(s, 1, fine - 1))
```

Output:

```
3
abc
```

Un programma può leggere se stesso o un altro sorgente con `readfile(ARGS[1])`: è esattamente il primo passo di un interprete self-hosted (capitolo 37).

---

14. Errori di esecuzione

Core non ha `try` / `catch` né valori `Err` (SPEC-CORE §9): un errore **termina il processo con exit code 1** e stampa un messaggio diagnostico su stderr. Quello che era già stato stampato su stdout resta.

```
println("prima")
println(non_esiste)
println("dopo")
```

```
stdout:  prima
stderr:  error: undefined variable non_esiste
exit:    1
```

Messaggi dell'interprete JS (stderr) e cause tipiche:

messaggio	causa
`parse error: unexpected token …` / `parse error: expected …`	sintassi non Core (es. `1.5`, `/`, `() -> …`, `end` mancante)
`error: undefined variable NOME`	nome mai assegnato, o funzione chiamata prima di essere definita
`error: not a function`	si chiama un valore che non è una funzione (`x = 5; x(1)`)
`error: arity mismatch`	numero di argomenti diverso dai parametri
`error: index out of range`	`xs[i]` con `i < 1` o `i > length(xs)`
`error: + type error` (e simili per `- * % < …`)	operandi di tipo sbagliato (`1 + "a"`)
`error: if cond` / `while cond` / `ternary cond` / `&& expects bool`	condizione non `Bool`
`error: div by zero` / `negative exponent`	`x % 0`, `2 ^ -1`
`error: unknown function NOME`	builtin riservato usato in modo non supportato
`cannot read file: …`	file inesistente o non leggibile
`usage: triad FILE [ARG...]` (exit 2)	nessun file passato

Il seed Windows non stampa messaggi dettagliati: esce con codice `1` (sintassi, variabile o funzione non definita, arità, lettura) o `2` (nessun argomento).

14.1 Errori come valori, in Core

Per propagare errori senza terminare il processo (come farà S1 per i programmi Full) si usa una convenzione con liste: `[:Err, kind, msg]`.

```
err(kind, msg) = [:Err, kind, msg]
is_err(v) = typeof(v) == :List && length(v) == 3 && v[1] == :Err
function div_intera(a, b)
    if b == 0
        return err(:domain, "divisione per zero")
    end
    q = 0
    while a >= b
        a -= b
        q += 1
    end
    q
end
r = div_intera(7, 0)
if is_err(r)
    println("errore ", r[2], ": ", r[3])
end
println(div_intera(17, 5))
```

Output:

```
errore domain: divisione per zero
3
```

Nota: `&&` cortocircuita, quindi `v[1]` è valutato solo se `v` è una lista di 3 elementi.

---

15. Interprete JS e seed Windows: differenze

Il contratto è `conformance/core/`: entrambi i runtime devono produrre gli stessi golden. Oggi l'interprete JS li passa tutti, il seed Windows i 33 casi elencati nel job `seed-windows` della CI. Fuori da quei casi i due runtime differiscono:

funzionalità	interprete JS (`seed/dev/triad_core.cjs`)	seed Windows (`seed.exe`)
interi, Bool, operatori, confronti, ternario	✅	✅
`if` / `elseif` / `else`, `while`, `let`, `begin`	✅	✅
funzioni forma breve e blocco, ricorsione	✅	✅ (max 4 parametri)
`return`, `break`, `continue`	✅	non elencati in `seed/windows-x86_64/README.md`
lambda	✅ più parametri	✅ **un solo parametro**
chiusure	condivise (mutazioni visibili, SPEC-CORE §5)	**per valore**, solo l'ultimo legame del frame
stringhe, `*`, `length(s)`	✅	✅
`string(x)`	✅ tutti i tipi	solo `Int` e `String`
simboli, `nothing`	✅	✅
liste	eterogenee, `xs[i]`, `xs[i] = v`, `length`, `push!`	solo interi; costruzione, stampa e `push!` (max 64 elementi)
`get(xs, i)`	non esiste	helper **non standard**, indici **da 0**
`-=`, `*=`, `Int`, `Symbol`, `typeof`	✅	non elencati nel README del seed
`ARGS`, primitivi di bootstrap	✅	non elencati nel README del seed
stampa di una funzione	`#lambda(x)`	`fun`
`triad -` (stdin)	✅	no

funzionalità	interprete JS (`seed/dev/triad_core.cjs`)	seed Windows (`seed.exe`)
messaggi d'errore	su stderr	solo exit code

Regola pratica: **scrivi codice che rispetta SPEC-CORE e verificalo con l'interprete JS**; se deve girare sul seed (come `selfhost/`), restringiti alla colonna di destra e verificalo su Windows.

---

16. Pattern idiomatici Triad-Core

16.1 Funzioni di ordine superiore scritte in Core

Core non ha `map` / `filter` / `reduce`: si scrivono una volta con `while` e lambda.

```
function mappa(f, xs)
    out = []
    i = 1
    while i <= length(xs)
        push!(out, f(xs[i]))
        i += 1
    end
    out
end
function filtra(p, xs)
    out = []
    i = 1
    while i <= length(xs)
        if p(xs[i])
            push!(out, xs[i])
        end
        i += 1
    end
    out
end
function riduci(f, acc, xs)
    i = 1
    while i <= length(xs)
        acc = f(acc, xs[i])
        i += 1
    end
    acc
end
xs = [1, 2, 3, 4, 5, 6]
println(mappa(x -> x * x, xs))
println(filtra(x -> x % 2 == 0, xs))
println(riduci((a, b) -> a + b, 0, xs))
```

Output:

```
[1, 4, 9, 16, 25, 36]
[2, 4, 6]
21
```

16.2 Record come liste taggate

Core non ha `struct`: un record è una lista con un simbolo come tag nella prima posizione.

```
punto(x, y) = [:Punto, x, y]
px(p) = p[2]
py(p) = p[3]
somma_punti(a, b) = punto(px(a) + px(b), py(a) + py(b))
p = somma_punti(punto(1, 2), punto(10, 20))
println(p, " ", p[1] == :Punto)
```

Output:

```
[:Punto, 11, 22] true
```

16.3 Ambiente come lista di associazioni

È la struttura usata dagli interpreti in `selfhost/`: una lista piatta `[nome1, valore1, nome2, valore2, …]` (in Core puoi anche usare coppie annidate).

```
function env_get(env, nome)
    i = 1
    while i < length(env)
        if env[i] == nome
            return env[i + 1]
        end
        i += 2
    end
    nothing
end
function env_set(env, nome, valore)
    i = 1
    while i < length(env)
        if env[i] == nome
            env[i + 1] = valore
            return env
        end
        i += 2
    end
    push!(env, nome)
    push!(env, valore)
end
env = []
env_set(env, :x, 10)
env_set(env, :y, 20)
env_set(env, :x, 11)
println(env, " ", env_get(env, :x), " ", env_get(env, :z))
```

Output:

```
[:x, 11, :y, 20] 11 nothing
```

16.4 Stato mutabile condiviso

Poiché una funzione nominata non può riassegnare un globale, lo stato condiviso si tiene in una lista (mutabile) o in una chiusura.

```
stato = [0]
function tick()
    stato[1] += 1
end
tick()
tick()
println(stato[1])
```

Output:

```
2
```

16.5 Costruire stringhe

```
function ripeti(s, n)
    out = ""
    while n > 0
        out = out * s
        n -= 1
    end
    out
end
function unisci(xs, sep)
    out = ""
    i = 1
    while i <= length(xs)
        if i > 1
            out = out * sep
        end
        out = out * string(xs[i])
        i += 1
    end
    out
end
println(ripeti("ab", 3))
println(unisci([1, 2, 3], ", "))
```

Output:

```
ababab
1, 2, 3
```

16.6 Stile

- Funzioni piccole con nomi chiari (in italiano, come nel resto del repository).

- Caso base esplicito in ogni ricorsione; per profondità grandi preferisci `while`.
- Cicli con contatori espliciti (`i = 1 … while i <= length(xs)`).
- Non usare `fact`, `rule`, `solve`, `struct` e le altre parole chiave di Full come nomi: il codice deve restare valido anche in Triad-Full.
- Scrivi `!io` sulle funzioni che stampano: Core lo ignora, Full lo richiede.

---

17. Esercizi guidati

Ogni esercizio ha una soluzione verificata (aprila solo dopo averci provato). Per controllare la tua soluzione: salvala in un file ed eseguila con `node seed/dev/triad_core.cjs`.

Esercizio 1 — Potenza e somme

Senza usare `^`, scrivi `potenza(b, e)` ricorsiva e `somma_fino(n)` (somma `1 + … + n`) con `while`. Stampa `potenza(3, 13)`, `potenza(2, 10)` e `somma_fino(100)`.

▶ Soluzione

Esercizio 2 — Numeri primi

Stampa la lista dei numeri primi minori di 30.

▶ Soluzione

Esercizio 3 — Inverti una lista

Scrivi `inverti(xs)` che restituisce una nuova lista con gli elementi in ordine inverso.

▶ Soluzione

Esercizio 4 — Ordinamento per inserzione

Ordina in loco `[5, 2, 9, 1, 7]`.

▶ Soluzione

Esercizio 5 — Contatore di parole

Conta le parole (separate da spazi) di `"il linguaggio Triad "` usando `strbyte`.

▶ Soluzione

Esercizio 6 — Un mini-tokenizer

Trasforma `"12+x*3"` in una lista di token `[tipo, testo]`, con tipi `:num`, `:id`, `:op`. È il primo passo di un interprete (vedi `selfhost/tokenizer.td`).

▶ Soluzione

Esercizio 7 — Valutare un'espressione

Usando i token dell'esercizio 6, scrivi un valutatore a discesa ricorsiva per `+` e `*` con precedenza, dove le variabili vengono da una lista di associazioni. Suggerimento: tieni la posizione corrente in una lista `pos = [1]` per poterla aggiornare dalle funzioni.

▶ Soluzione

Parte III — Triad-Full (lo standard completo)

Questa parte descrive il linguaggio definito da `SPEC.md`. **Non è ancora eseguibile** dal runtime attuale: serve per conoscere lo standard, scrivere codice pronto per S1 e capire i golden di `conformance/cases/`. Ogni capitolo indica la sezione di SPEC da cui deriva. Gli output marcati **Output (golden Full):** provengono dai golden congelati.

18. Lessico e tipi Full (SPEC §1, §4)

Full estende il lessico di Core con:

elemento	forma	esempi
identificatore	come Core, anche lettere non ASCII; `!` finale se non seguito da `=`	`età`, `push!`
variabile logica	`?` **attaccato** a un nome	`?x`, `?who`
variabile anonima	`?_` (ogni occorrenza è nuova)	`parent(?x, ?_)`
`Float`	`3.14`, `1e-9`	
interpolazione	`"x = $x"`, `"$(a + b)"`; escape aggiuntivo `\$`	
commenti annidati	`#= a #= b =# c =#`	
macro	`@nome(...)`	`@unless(c, e)`

Parole chiave: `function struct macro end if elseif else for in while let begin return break continue fact rule solve quote try catch with requires ensures true false nothing isa is`. Contestuali dopo `solve`: `limit depth steps trace`.

Regole dei due punti `:`

- `1:n`, `xs[2:end]`: `:` attaccato a un valore, senza spazi → range;
- `f(:a)`, `x = :a`: `:` non preceduto da un valore e seguito da un nome → simbolo;
- `:( … )` → codice quotato;
- `c ? a : b`: `:` circondato da spazi → ternario.

Regola del `?`: `?x` attaccato = variabile logica; `?` con spazio = ternario.

Gli a capo chiudono un'istruzione, tranne dentro `( )`, `[ ]` e `:( )`, e subito dopo un operatore binario o una virgola di `rule`.

Tipi Full (oltre a quelli di Core):

tipo	valori
`Int`, `Float` (sottotipi di `Number`)	`42`, `3.5`
`Expr`	codice quotato (`Node`)
`Term`, `Cons`	termini e pattern lista logici
`Solution`	`{who = :dave}`
`Err`	errore come dato
`Type`	`typeof(3) == Int`
struct utente	`Point(3.0, 4.0)`
`Any`	supertipo di tutto, usato nelle firme

Variabili tipate:

```
x::Int = 5
x = 6          # bene
x = 5.5        # Err(:type): il nome mantiene il tipo dichiarato
y::Float = 1.0
y = 2          # 2.0: conversione Int → Float
```

Il vincolo nasce con la dichiarazione e vive nell'ambiente in cui il nome è stato creato.

---

19. Funzioni tipate e multiple dispatch (SPEC §5)

```
sq(x) = x * x
area(n::Number) = :number
area(n::Int) = :int

function dist(a::Point, b::Point)::Float
    requires a.x != b.x || a.y != b.y
    sqrt((a.x - b.x)^2 + (a.y - b.y)^2)
end

function +(a::Point, b::Point)      # gli operatori sono funzioni r
    Point(a.x + b.x, a.y + b.y)
end
```

Regole:

- vince il metodo **più specifico** tra gli applicabili (`area(1) == :int` , `area(1.5) == :number`);
- nessun metodo applicabile → `Err(:method)` (o fallback builtin per gli operatori numerici);
- più metodi ugualmente specifici → `Err(:ambiguous)` ;
- il tipo di ritorno `::R` è controllato all'uscita (con conversione Int→Float);
- funzioni e operatori sono valori: `reduce(+, xs)` , `map(sq, xs)` ;
- lambda tipate: `function (x::Int) x end` → `f(2.0)` è `Err(:type)` .

---

20. Struct (SPEC §4)

```
struct Point
    x::Float
    y::Float
end

p = Point(3, 4)      # costruttore posizionale, Int → Float
p.x                  # 3.0
typeof(p) == Point   # true
p isa Point          # true
```

- Immutabili; campi tipizzati.
- Campo inesistente → `Err(:field)`; tipo sbagliato → `Err(:type)`.
- In Core lo stesso concetto si rappresenta con una lista taggata (§16.2).

---

21. Controllo di flusso Full: `for`, range, comprehension, `try` (SPEC §2)

```
s = 0
for i in 1:10
    s += i
end                                 # s == 55

xs = [1, 2, 3]
xs[end]                             # 3
xs[2:end]                           # [2, 3]
[x^2 for x in 1:4 if x > 2]         # [9, 16]
x = 3; x > 2 ? :big : :small        # :big

e = try div(1, 0) end               # l'errore diventa il valore
r = try div(1, 0) catch err; 0 end  # con catch: err è l'Err, il v
```

Altri valori attesi (dai vecchi test dell'implementazione Julia):

```
x = 2; y = x * 21; y                              # 42
7 / 2                                             # 3.5
"ab" * "cd"                                       # "abcd"
let a = 1, b = a + 1; a + b; end                  # 3
if 1 > 2; :a; elseif 2 > 1; :b; else; :c; end     # :b
sq(x) = x * x; map(sq, [1, 2, 3])                 # [1, 4, 9]
reduce(+, [1, 2, 3, 4])                           # 10
filter(x -> x % 2 == 0, 1:6)                      # [2, 4, 6]
function fact_(n::Int)::Int
    if n <= 1; return 1; end
    n * fact_(n - 1)
end; fact_(10)                                    # 3628800
```

`requires c` al top level è un'asserzione; `ensures c` è ammesso solo dentro `function`.

---

22. Logica: fatti, regole, `solve` (SPEC §6)

```
fact parent(:alice, :bob)
fact parent(:alice, :carol)
fact parent(:bob, :dave)

rule grandparent(?x, ?z) :- parent(?x, ?y), parent(?y, ?z)
rule ancestor(?a, ?d) :- parent(?a, ?d)
rule ancestor(?a, ?d) :- parent(?a, ?x), ancestor(?x, ?d)
```

`solve` restituisce una **lista di `Solution`** (è un valore, non stampa nulla):

```
sols = solve grandparent(:alice, ?who) limit 5
[s.who for s in sols]                       # [:dave]
[s.d for s in solve ancestor(:alice, ?d)]
```

- Opzioni: `limit n`, `depth d`, `steps s` (default 1 000 000), `trace`.
- Un predicato usato come funzione è un `Bool`: `grandparent(:alice, :dave)` → `true`; si può usare in `if`.
- Goal: `p(t…)`, `g1 && g2`, `(g1, g2)`, `g1 || g2`, `!g` / `not(g)`, `a = b` (unificazione), `?y is espr`, confronti, funzione Triad che restituisce `Bool`, `true` / `false`.
- Occurs-check attivo: `solve ?x = f(?x)` non ha soluzioni (non va in ciclo).
- Liste stile Prolog nei pattern: `[?h | ?t]`.
- `vars(s)` restituisce i nomi legati in una `Solution`; si accede con `s.nome` o `s[:nome]`.

Esempi:

```
# percorso con costo
fact edge(:a, :b, 2)
fact edge(:b, :c, 3)
rule path(?x, ?y, ?c) :- edge(?x, ?y, ?c)
rule path(?x, ?y, ?c) :- edge(?x, ?z, ?c1), path(?z, ?y, ?c2), ?c i
(solve path(:a, :c, ?costo))[1].costo      # 5

# append alla Prolog
rule app([], ?l, ?l)
rule app([?h | ?t], ?l, [?h | ?r]) :- app(?t, ?l, ?r)
(solve app([1, 2], [3], ?r))[1].r          # [1, 2, 3]
length(solve app(?a, ?b, [1, 2, 3]))       # 4

# negazione per fallimento
fact parent(:tom, :bob)
persone = [:tom, :bob]
foglie = filter(p -> !parent(p, ?_), persone)   # [:bob]
```

23. Destrutturazione (SPEC §2)

Pattern di lista con **arità esatta**; le parentesi attorno al pattern sono obbligatorie.

```
(a, b) = [1, 2]                # a == 1, b == 2
e = try (a, b) = [1] end       # e.kind == :bounds
e = try (a, b) = [1, 2, 3] end  # e.kind == :bounds

xs = []
for (k, v) in [[1, 2], [3, 4]]
    push!(xs, k + v)
end                            # xs == [3, 7]
```

- Un elemento non lista o di arità diversa nel `for` → `Err(:bounds)` o `Err(:type)`.
- `function f((a, b))` definisce **due parametri** `a` e `b` (i parametri sono appiattiti): `f(10, 20) == 30`. Per destrutturare un singolo argomento lista usa `(a, b) = pair` nel corpo.
- I pattern annidati ricorsivi non sono ancora previsti.

---

24. Errori come dati (SPEC §8)

Un errore è un valore `Err(kind, msg, data)`; non diventa un'eccezione dell'host salvo che non sia gestito al top level (allora il processo esce con codice 1 e stdout vuoto per l'istruzione fallita: è il caso di conformance `errore`).

```
e = try div(1, 0) end     # Err(:domain, ...)
e.kind                    # :domain
e.msg                     # messaggio
iserr(e)                  # true
7.0 / 0                   # Inf: la divisione Float non è un errore
throw(err(:mio, "ops", 42))
error("boom")             # Err(:error, ...)

describe(e::Err) = string("errore ", e.kind, ": ", e.msg)
describe(x::Any) = string("valore ", x)
for r in [try div(10, d) end for d in [2, 0, 5]]
    println(describe(r))
end
```

Kinds standard: `:syntax`, `:undef`, `:type`, `:method`, `:ambiguous`, `:field`, `:bounds`, `:domain`, `:instantiation`, `:contract`, `:budget`, `:effect`, `:host`.

`Err(:syntax)` porta in `data` la posizione (`e.data.line`, `e.data.col`).

---

25. Contratti (SPEC §9)

```
function safe_div(a::Number, b::Number)::Float
    requires b != 0
    ensures abs(result * b - a) < 1e-9
    a / b
end

safe_div(6, 3)            # 2.0
e = try safe_div(1, 0) end
e.kind                    # :contract
e.data.clause             # :requires
e.data.fn                 # :safe_div
```

- `requires c` è controllata dove compare (inizio del corpo, oppure al top level come asserzione).
- `ensures c` è controllata all'uscita, con `result` legato al valore restituito, dopo il controllo del tipo di ritorno.
- Violazione → `Err(:contract, …)` con `data.clause`, `data.fn`, `data.cond`, `data.result`.
- I contratti sono **puri**: un effetto dentro un contratto → `Err(:effect)`.
- `ensures` fuori da `function` → `Err(:syntax)`.

---

26. Budget (SPEC §10)

```
r = with budget(steps = 10_000)
    while true end
end
r.kind                    # :budget — il ciclo infinito diventa un
```

- Un passo = una chiamata, un'iterazione di ciclo, un passo di risoluzione.

- I budget si annidano: vale il più stretto; solo il budget del `with` corrente diventa valore, quello esterno si propaga.
- `solve … steps N` è il budget di una singola query.
- La ricorsione infinita non fa stack overflow: finisce in `Err(:budget)` (stack esplicito).

---

27. Tracce (SPEC §11)

La traccia è un **valore**, non output.

```
fib(n::Int) = n < 2 ? n : fib(n - 1) + fib(n - 2)
t = with trace
    fib(3)
end
t.result      # 2
t.trace       # [Event(port=:call, fn=:fib, args=[3], depth=0), …]

t = solve path(:a, :c) trace
t.trace       # porte Byrd: :call, :exit, :redo, :fail, :depth
```

Gli eventi sono record con `port`, `goal`/`fn`, `depth` (e `value`/`args` per le funzioni): si filtrano come qualsiasi valore.

---

28. Effetti dichiarati (SPEC §12)

Le funzioni sono pure per default. Effetti: `io` (`println`, `print`) e `state` (`fact`/`rule` dentro una funzione).

```
function saluta(nome::String) !io
    println("ciao, ", nome)
end

function impara(x) !state
    fact visto($x)
end
```

- Controllo dinamico: una funzione pura che chiama una `!io` o `println` → `Err(:effect)`.
- Il top level permette tutti gli effetti; le lambda ereditano quelli del contesto.
- Macro e contratti sono puri; leggere la KB (`solve`, predicati) è puro.
- Mutare liste o variabili locali non è un effetto.

---

29. Codice come dato, albero uniforme e macro (SPEC §3, §7)

29.1 L'albero uniforme

Il parser produce un solo tipo di nodo, `Node(head, args)`: è lo stesso albero che il kernel esegue, che `:( … )` restituisce e che `--tree` stampa come S-expression.

sintassi	nodo
`f(a, b)`, `a + b`	`(call f a b)`, `(call + a b)`
`a && b`, `a || b`	`(and a b)`, `(or a b)`
`x = e`	`(assign x e)`
`:alice`	`(sym alice)`
`a.x`, `xs[i]`	`(dot a x)`, `(index xs i)`
`[a, b]`, `[h | t]`	`(list a b)`, `(cons (list h) t)`
`[e for x in xs if c]`	`(comprehension e x xs c)`
`x -> e`	`(lambda (tuple x) e)`
`x::T`	`(typed x T)`
`"a $x"`	`(string "a " x)`
`if c a else b end`, `c ? a : b`	`(if c (block a) (block b))`
`while`, `for`, `let`, `begin`	`(while c body)`, `(for x iter body)`, `(let (tuple …) body)`, `(block …)`
`function f(a::T)::R !io … end`	`(function f (tuple (typed a T)) R (tuple io) body)`
`struct P; x::Float; end`	`(struct P (tuple (typed x Float)))`
`macro m(a) … end`, `@m(e)`	`(macro m (tuple a) body)`, `(macrocall m e)`
`:(e)`, `quote … end`, `$e`	`(quasi e)`, `(quasi (block …))`, `(unquote e)`
`fact t`, `rule h :- g1, g2`	`(fact t)`, `(rule h g1 g2)`
`solve g limit 5 trace`	`(solve g (tuple (assign limit 5) (assign trace true)))`

sintassi	nodo
`try b catch e h end`	`(try b e h)`
`with budget(steps = N), trace … end`	`(with (tuple (call budget (assign steps N)) trace) body)`

29.2 Quote, unquote, eval

```
c = :(a.x + f(b, 2))
c.head        # :call
sexpr(c)      # "(call + (dot a x) (call f b 2))"

n = 7
quadrato = :($n * $n)
eval(quadrato)                  # 49
parse("if x > 0; :pos; else; :neg; end")   # albero
unparse(parse_program("1:n"))   # "1:n"
Expr(:call, :f, 1)              # costruisce un Node
```

29.3 Macro

Le macro ricevono codice e restituiscono codice; l'espansione è pura e **non igienica**: usa `gensym()` per nomi freschi.

```
macro unless(c, body)
    :(if !$c; $body; end)
end

macro swap(a, b)
    tmp = gensym(:tmp)
    quote
        $tmp = $a
        $a = $b
        $b = $tmp
    end
end

macro time_it(label, e)
    :(begin
        r = $e
        println($label, " -> ", r)
        r
    end)
end

@swap(x, y)
@unless(x < y, println("x non è minore di y"))
@time_it("somma", reduce(+, 1:100))     # stampa "somma -> 5050"
```

29.4 Codice che genera codice

```
function poly(coeffs)
    acc = :(0)
    i = 0
    for c in coeffs
        acc = :($acc + $c * x ^ $i)
        i += 1
    end
    acc
end
p = poly([1, 2, 3])
x = 2
eval(p)        # 17
```

---

30. Builtin Full (SPEC §13)

ambito	funzioni
aritmetica e confronto	`+ - * / ^ % == != < <= > >= !`, `abs sqrt floor ceil round exp log sin cos div mod min max`; `*` concatena stringhe
liste	`length isempty first last rest reverse sort sum push! pop! append! vcat collect`, `a:b`, `x in xs`
ordine superiore	`map filter reduce any all count`
stringhe e output	`string repr println print`
tipi	`typeof`, `isa`, `Int Float String Symbol`
errori	`err iserr throw error`
codice	`eval parse unparse sexpr Expr gensym`
logica	`vars(sol)`

Di questi, Core fornisce solo `+ - * % ^`, confronti, `&& || !`, `length`, `push!`, indicizzazione, `*` su stringhe, `Int`, `string`, `Symbol`, `print`, `println`, `typeof` (SPEC-CORE §7).

---

31. Riga di comando e REPL (SPEC §14)

Interfaccia normativa di `triad` (oggi il runtime implementa solo `triad FILE [ARG...]` e, nell'interprete JS, `triad -`):

```
triad                           # REPL
triad FILE [ARG...]             # esegue FILE; ARGS == [ARG...]
triad -l FILE                   # carica FILE e apre il REPL
triad -i FILE                   # esegue FILE e poi apre il REPL
triad -e CODE                   # esegue CODE
triad -E CODE                   # esegue CODE e stampa il valore
triad --tree FILE               # stampa l'albero uniforme come S-
triad -q                        # REPL senza banner
triad -h, --help
triad -v, --version
triad -                         # legge lo script da stdin
```

- L'estensione `.triad` / `.td` si può omettere; lo shebang `#!` è ignorato.
- Exit: `0` successo, `1` errore non gestito (stampato su stderr), `2` uso non valido.

REPL (stile Prolog):

```
triad> 1 + 2
3
triad> ?- ancestor(:tom, ?d)
?d = :bob ;
?d = :liz.
triad> parent(:jim, ?x)
false.
```

- I blocchi aperti restano in attesa fino a `end`.
- `?- goal` (o un goal su un predicato della KB) esegue una query; dopo ogni soluzione `;` / `n` /spazio = successiva, `a` = tutte, invio/ `.` = stop.
- Comandi: `:load FILE…` (`:l`), `:reload` (`:r` , `make`), `:listing [nome|nome/arità]` (`:ls`), `:tree CODICE` , `:reset` , `:help` (`:h`), `:quit` (`:q` , `halt` , Ctrl-D).

---

32. Esempi Full con output golden

I file sono in `examples/` (eseguibili quando S1 sarà pronto); gli output sono i golden congelati in `conformance/cases/`.

file	copre	caso golden
`examples/assaggio.td`	struct, dispatch, logica, macro, contratti	`assaggio`
`examples/famiglia.td`	fatti/regole, ricorsione, liste Prolog, negazione	`famiglia`
`examples/garanzie.td`	errori, contratti, budget, tracce, effetti	`garanzie_core`
`examples/macro.td`	quote, eval, macro, gensym	`macro`
`examples/tour.td`	tutti i paradigmi	`tour`
`examples/hello_shebang.td`	shebang, `ARGS`, effetti	`hello_shebang`
`examples/benvenuto.triad`	estensione canonica, `Float`, `solve`	—

32.1 `assaggio.td`

```
struct Point
    x::Float
    y::Float
end

function dist(a::Point, b::Point)::Float
    requires a.x != b.x || a.y != b.y
    sqrt((a.x - b.x)^2 + (a.y - b.y)^2)
end

fact parent(:alice, :bob)
fact parent(:bob, :dave)
rule grandparent(?x, ?z) :- parent(?x, ?y), parent(?y, ?z)

result = solve grandparent(:alice, ?who) limit 5
println("nipoti: ", result)     # [{who = :dave}]
println("primo: ", result[1].who) # :dave
```

Output (golden Full) del file completo:

```
nipoti di alice: [{who = :dave}]
primo: dave
quanti: 1
dist(p, q) = 5.0
dist(p, p) -> Err(:contract, "requires violata in dist: a.x != b.x
tipo: contract
almeno un nipote
alice è nonna di dave? true
```

32.2 `famiglia.td`

Output (golden Full):

```
discendenti di tom: [:bob, :liz, :ann, :pat, :jim]
fratelli di ann: [:pat]
madri: ["pat -> jim"]
senza figli: [:liz, :ann, :jim]
over 50: [:tom, :bob]
modi di spezzare [1, 2, 3]:
  [] ++ [1, 2, 3]
  [1] ++ [2, 3]
  [1, 2] ++ [3]
  [1, 2, 3] ++ []
figli di bob: [:ann, :pat]
```

32.3 `garanzie.td`

Output (golden Full, traccia abbreviata):

```
2.5
violazione: contract — requires violata in safe_div: b != 0
clausola: requires, funzione: safe_div
valore 5
errore domain: divisione intera per zero
valore 2
valore 42
errore input: età non valida
catturato mio con dati [1, 2, 3]
ciclo infinito -> budget
solve infinito -> budget
fib(3) = 2 in 10 eventi
chiamate: [[3], [2], [1], [0], [1]]
  call path(:a, :c) @0
  call edge(:a, :c) @1
  fail edge(:a, :c) @1
  …
ciao, Triad
effetto non dichiarato -> println richiede !io ma rumorosa non lo d
imparato: [:gatto]
```

32.4 `macro.td`

Output (golden Full):

```
codice:  :(a.x + f(b, 2))
head:    call
args:    [:+, :(a.x), :(f(b, 2))]
sexpr:   (call + (dot a x) (call f b 2))
:(7 * 7) = 49
parse:   (if (call > x 0) (block (sym pos)) (block (sym neg)))
unparse: if x > 0; :pos; else; :neg; end
dopo @swap: x = 2, y = 1
x non è minore di y
somma -> 5050
polinomio: :(0 + 1 * x ^ 0 + 2 * x ^ 1 + 3 * x ^ 2)
p(2) = 17
```

32.5 Contratti e budget insieme

```
function safe_div(a::Number, b::Number)::Float
    requires b != 0
    ensures abs(result * b - a) < 1e-9
    a / b
end
e = try safe_div(1, 0) end
e.kind          # :contract
r = with budget(steps = 1000)
    while true end
end
r.kind          # :budget
```

Parte IV — Il codice del progetto

Questa parte spiega come è fatto il repository e come si scrive (o si modifica) ogni tipo di codice che contiene.

33. Mappa del repository

percorso	linguaggio	cosa contiene
`SPEC.md`, `SPEC-CORE.md`, `VERSION`, `CHANGELOG.md`	Markdown	lo standard (Full e Core), versione, storia
`BOOTSTRAP.md`, `STATUS.md`, `TASK.md`, `plan.md`	Markdown	piano S0→S4, stato, backlog e task
`conformance/`	Triad + Node	casi golden (`cases/` Full, `core/cases/` Core) e runner `run.mjs`
`seed/windows-x86_64/`	codice macchina	`seed.hex` (sorgente), `seed.exe` (decodificato), README con layout
`seed/linux-x86_64/`	codice macchina	seed ELF E0 (`seed.hex`, `seed`)
`seed/dev/`	Node.js	generatori del seed (`gen_seed.js`, `gen_seed_linux.js`) e interprete di riferimento `triad_core.cjs`
`selfhost/`	Triad-Core	S1: interprete Triad scritto in Triad-Core (in corso)
`examples/`	Triad-Full	programmi di esempio dello standard
`cli/`	Node.js (ESM)	`triad-cli` : CLI di sviluppo (commander, execa, chalk)
`bin/`	sh / cmd	launcher `triad`, `triad.cmd`, `triad-cli`, `triad-cli.cmd`
`ai/`	Triad + Node	benchmark di generazione codice per LLM, prompt, task con golden

percorso	linguaggio	cosa contiene
`editors/`	JSON / Vim script	grammatica VS Code (TextMate), sintassi Vim, voce Linguist
`docs/`	HTML/CSS/JS	sito GitHub Pages; carica i Markdown del repo (questa wiki compresa)
`obsidian-triad/`	Markdown	vault Obsidian con indice dei file e grafo del progetto
`.github/workflows/`	YAML	CI (`ci.yml`) e pubblicazione del sito (`pages.yml` , `static.yml`)
`test_*.td` , `min_*.td` (root)	Triad-Core	piccoli probe usati durante lo sviluppo del seed (es. `test_get.td` in CI)

Convenzioni comuni:

- documentazione, commenti e messaggi sono in **italiano**;
- fine riga **LF** ovunque, tranne `*.cmd` (CRLF), imposto da `.gitattributes` ;
- `.gitignore` esclude `*.out` con l'eccezione `!conformance/**/expected.out` .

---

34. L'interprete di riferimento JS (`seed/dev/triad_core.cjs`)

È un interprete Triad-Core completo in un solo file CommonJS (~1000 righe, nessuna dipendenza). È lo strumento con cui si verifica la conformance Core su Linux/macOS ed è il modo più rapido per capire la semantica di Core leggendo codice.

34.1 Architettura

```
sorgente ──tokenize()──▶ token ──Parser──▶ AST {kind: …} ──evalStmt
```

1. **Valori** (in cima al file): oggetti `{t, v}` costruiti da `I` (Int, `BigInt` con `wrap()` a 64 bit), `B`, `S`, `Y` (Symbol), `L` (List), `F` (funzione: `params`, `body`, `env`, `isLambda`) e la costante `NOTHING`. I builtin sono `{t:'builtin', name}`.
2. **`tokenize(src)`**: produce token `SEP` (a capo o `;`), `NUM`, `STR`, `SYM`, `IDENT`, `OP`, `LPAREN` / `RPAREN`, `LBRACKET` / `RBRACKET`, `COMMA`, `QMARK`, `COLON`, `EOF`. Commenti e spazi sono scartati qui.
3. **`class Parser`**: discesa ricorsiva che segue la precedenza del §7.1: `parseProgram → parseStmt → parseExpr → parseAssign → parseTernary → … → parsePostfix → parsePrimary`, più `parseBlock(terminators)` per i corpi fino a `end` / `else` / `elseif`. Nodi prodotti: `program`, `funcdef`, `return`, `break`, `continue`, `exprstmt`, `num`, `str`, `sym`, `bool`, `nothing`, `var`, `list`, `lambda`, `assign`, `indexassign`, `binary`, `unary`, `ternary`, `call`, `index`, `if`, `while`, `let`, `begin`.
4. **Ambienti**: `makeFrame(parent, owner, isLambda)` crea un frame con una `Map` di legami; `lookup` risale la catena. `writeVar` implementa le regole di scope del §10.3: aggiorna un legame dello stesso "proprietario" (funzione), oppure, dentro una lambda, un legame esterno; altrimenti crea un locale.
5. **Valutazione**: `evalStmt`, `evalExpr` → `evalExprBinary` → `evalExprRest` → `evalExprBlocks`. `return`, `break`, `continue` sono eccezioni JS (`ReturnSignal`, `BreakSignal`, `ContinueSignal`) catturate dalla chiamata o dal `while`.
6. **Builtin**: insieme `BUILTIN_NAMES` + `callBuiltin(name, args, env)`; stampa con `printValue` (simboli senza `:` solo al top level) e conversione con `stringOf`.
7. **`main()`**: legge il file (o stdin con `-`), lega `ARGS`, esegue; errori di parsing → `parse error: …`, errori di esecuzione → `error: …`, entrambi con exit 1; nessun file → exit 2.

34.2 Esempio: aggiungere un builtin

Supponiamo di voler sperimentare un builtin `abs` (non è in Core: va prima proposto nello standard, vedi §43). Servono due modifiche:

```
// 1. registrare il nome
const BUILTIN_NAMES = new Set(['print','println','length','push!','
  'typeof','readfile','strlen','strbyte','strsub','exit','abs']);

// 2. implementarlo in callBuiltin, prima del throw finale
  if (name === 'abs') {
    if (args.length !== 1 || args[0].t !== 'int') throw new Error('
    return I(args[0].v < 0n ? -args[0].v : args[0].v);
  }
```

Poi: un caso in `conformance/core/cases/` (§38) e `node conformance/run.mjs`. Ricorda che gli `Int` sono `BigInt` (`0n`, `1n`) e che i risultati vanno costruiti con gli helper (`I`, `B`, `S`, `Y`, `L`, `NOTHING`).

34.3 Esempio: aggiungere sintassi

1. se serve un nuovo token, estendi `tokenize` (es. un operatore in `twoOps`);
2. aggiungi il livello di precedenza nel `Parser` producendo un nuovo `kind`;
3. gestisci il `kind` in `evalExpr*`;
4. aggiorna `SPEC-CORE.md` (grammatica §4 e semantica §5) e aggiungi un caso Core.

Regola: l'interprete JS deve restare **fedele a SPEC-CORE**; non aggiungere estensioni che il seed e S1 non dovrebbero avere.

---

35. Il seed S0 Windows e il suo generatore

35.1 Cos'è

`seed/windows-x86_64/seed.exe` è un PE32+ x86-64 di **25.088 byte** che importa solo `KERNEL32.dll` (`GetStdHandle`, `GetCommandLineA`, `CreateFileA`, `ReadFile`, `WriteFile`, `CloseHandle`, `ExitProcess`,

`VirtualAlloc`). Non usa libc né runtime. La **fonte** è il dump `seed.hex` (SHA-256 verificato dalla CI: `3A75A21E2F47D8DC4C88D4BC2580A85D5349BCDF7550E5FFA16E2C0AC7CA8C11`).

Il seed **non costruisce un AST**: interpreta direttamente il testo sorgente, con un cursore (`rsi`) che scorre i byte; i corpi di funzioni e lambda sono puntatori nel sorgente che vengono rieseguiti a ogni chiamata.

35.2 Convenzioni interne

registro	ruolo
`r14`	base della memoria `VirtualAlloc` (sorgente a `+0`, ambiente a `+0x10000`, funzioni a `+0x20000`, heap a `+0x30000`)
`r15`	ambiente variabili (entry da 24 byte: nome ptr, len, valore)
`rbx`	contatore variabili
`r13`	tabella funzioni (entry da 128 byte)
`r12`	handle di stdout
`rsi`	cursore nel sorgente

Valori taggati a 2 bit: `Int = n << 2`, `false = 1`, `true = 5`, chiusura `= ptr | 2`, oggetti heap (stringhe `[tipo][len][byte…]`) con tag 3. Il layout completo (header PE, IAT, mappa delle etichette `parse_program`, `st_if`, `parse_primary`, `env_find`, …) è in `seed/windows-x86_64/README.md`.

35.3 Il generatore `seed/dev/gen_seed.js`

Scrivere migliaia di byte a mano è impraticabile: il generatore è un **assembler-lite** in JavaScript che emette i byte e risolve le etichette. Sezioni del file:

sezione	cosa
`emitter`	`B(...)` emette byte, `B32(v)` un dword, `L(nome)` definisce un'etichetta, `reloc` / `je` / `jne` / `jmp` / `callL` emettono salti con patch relative, `callIat(i)` chiama l'import `i`
helper di istruzione	funzioni con il nome dell'istruzione (`incRsi` , `cmpByteRsiImm('(')` , `testRax` , `leaRdxR14(d)` , …) che emettono i byte x86-64 corrispondenti
`PE/data layout`	costanti `IAT = 0x4000` , `SEC_RAW` , `HEADERS`
`main`	prologo, lettura della riga di comando, apertura/lettura del file
`parser`	il cuore dell'interprete: statement, espressioni, funzioni, lambda
`ambiente`	`env_find` , `env_store` , `env_assign`
`exit labels`	`fail` , `read_fail` , `open_fail` , `usage`
`PE builder`	scrive header DOS/PE/COFF, sezione `.text` , tabelle import, e il dump esadecimale

Esempio di stile (dal `main`): saltare il nome del programma nella riga di comando.

```
cmpByteRsiImm(0x22); jne('skip_plain');   // il path inizia con " ?
L('q_loop');
incRsi();
cmpByteRsiImm(0); je('no_arg');           // fine stringa: nessun a
cmpByteRsiImm(0x22); jne('q_loop');
```

35.4 Flusso per modificare il seed

1. modifica `seed/dev/gen_seed.js` (aggiungi un'etichetta, un ramo nel dispatcher, …);
2. rigenera il dump: `node seed/dev/gen_seed.js seed/windows-x86_64/seed.hex` (oppure `bin/triad-cli seed generate`

`windows`, che stampa anche lo SHA-256);

3. decodifica su Windows: `certutil -decodehex -f seed\windows-x86_64\seed.hex seed\windows-x86_64\seed.exe`;
4. esegui i casi Core sul seed e il probe `test_get.td`;
5. aggiorna **insieme**: hash in `.github/workflows/ci.yml` (job `seed-windows`, step `Hash check`), `seed/windows-x86_64/README.md`, `seed/dev/README.md`, `STATUS.md`; se un nuovo caso Core passa sul seed, aggiungilo all'array `$cases` del job.

Trappole note (da `STATUS.md`): il codice cresce e può sovrapporsi ai dati (i nomi import ora partono da RVA `0x5000`); registri usati prima di essere inizializzati (bug `pbs_int`); ordine degli argomenti sullo stack nelle chiamate builtin (`get`). Defender può segnalare il PE come falso positivo (la CI aggiunge un'esclusione).

---

36. Il seed Linux (E0)

`seed/linux-x86_64/seed` è un ELF64 statico di **173 byte** generato da `seed/dev/gen_seed_linux.js`: entry `0x400078`, un solo `PT_LOAD` R|X, syscall dirette `write` / `exit`. Stampa `Triad seed Linux OK` ed esce con 0. È il primo passo della milestone M7; E1/E2/E3 (lettura file, interprete) sono da fare, riusando la logica del seed Windows con un layer `os_*` diverso.

```
node seed/dev/gen_seed_linux.js seed/linux-x86_64/seed.hex   # veri
xxd -r -p seed/linux-x86_64/seed.hex seed/linux-x86_64/seed && chmo
./seed/linux-x86_64/seed                                     # Tria
```

---

37. Self-hosting: S1 in `selfhost/`

S1 è l'interprete di tutto Triad scritto in Triad-Core ed eseguito dal seed. Oggi è allo stadio di prototipi separati, ognuno con un piccolo test in fondo al file:

file	cosa prova
`main.td`	ambiente inline come lista piatta `[chiave, valore]`; smoke test in CI (atteso `Value: 10`)
`value.td`	valori taggati a 2 bit simulati con interi (`make_int`, `make_bool`, `as_int`)
`tokenizer.td`	`next_token(chars, pos)` su un sorgente codificato come lista `[lunghezza, byte…]`; token `[tipo, valore, pos]`
`parser.td`	riconoscimento delle keyword `if` / `end` dai codici ASCII
`eval.td`, `program.td`	valutazione di token numerici, identificatori, assegnamenti e `println`
`env.td`, `heap.td`, `ast.td`, `interpreter.td`	stub dell'ambiente, dell'allocatore bump e dell'AST

37.1 Come si scrive codice S1 oggi

Il codice S1 deve girare sul **seed Windows**, quindi segue le restrizioni della colonna destra del capitolo 15:

- usa `get(xs, i)` con indici **da 0** invece di `xs[i]` (non standard: andrà sostituito con l'indicizzazione Core quando il seed la supporterà);
- liste solo di interi: le stringhe sorgente si rappresentano come liste di codici ASCII, con la lunghezza nella posizione 0;
- `push!` restituisce la lista: si scrive `env = push!(env, v)`;
- costanti per i tipi di token (`0` EOF, `1` NUMBER, `2` IDENTIFIER, …) documentate in un commento in testa al file;
- funzioni con al massimo 4 parametri, lambda a un parametro, niente cattura multi-binding.

```
# stile S1: ambiente piatto e lookup con get (0-based)
env = []
env = push!(env, 1)      # chiave
env = push!(env, 10)     # valore
if get(env, 0) == 1
  println("Value: " * string(get(env, 1)))
end
```

Su Linux questi file **non girano** con l'interprete JS (che non ha `get`): vanno provati su Windows con `seed\windows-x86_64\seed.exe selfhost\main.td`.

37.2 Dove va S1

Le task S1.3–S1.15 sono in `TASK.md`: valori taggati, tokenizer e parser inline, eval di espressioni, `let`, programmi completi, funzioni e chiamate, chiusure, e infine far girare la conformance sopra seed+S1. Gli esercizi 6 e 7 del capitolo 17 sono una versione in Core standard degli stessi passi.

---

38. Conformance: leggere, scrivere e verificare casi

38.1 Formato

```
conformance/core/cases/<nome>/      (oppure conformance/cases/<nome
  program.td        programma Triad eseguito come script
  args.txt          opzionale: un argomento per riga (ARGS)
  expected.out      stdout atteso, byte per byte, fine riga LF
  expected.golden   copia di expected.out (fallback del runner)
  expected.code     exit code atteso
```

Un caso passa se stdout e codice di uscita coincidono; stderr non è confrontato. Niente stdin né input interattivo.

38.2 Il runner `conformance/run.mjs`

```
node conformance/run.mjs                                # Core (defau
node conformance/run.mjs --cases conformance/cases      # Full (oggi
node conformance/run.mjs --skip liste,valori            # salta casi
node conformance/run.mjs --update                       # RIGENERA i
```

Runtime scelto, in ordine: variabile `TRIAD_SEED`; su Linux/macOS l'interprete JS; su Windows `bin/triad-native.exe`, poi `seed/windows-x86_64/seed.exe`. Esempio per provare il seed con un percorso esplicito: `TRIAD_SEED=/percorso/seed node conformance/run.mjs`.

Messaggi:

- `PASS nome` / `FAIL nome (exit X, atteso Y, stdout N byte, attesi M)`;
- `golden mancante`: il checkout non ha `expected.out` (di solito basta `git pull`).

38.3 Aggiungere un caso Core

1. scegli un nome descrittivo in italiano (`liste_annidate`, `stringhe_vuote`, …);
2. scrivi `program.td` usando **solo** SPEC-CORE §2–§7 (niente primitivi di bootstrap: devono girare anche su Full);
3. scrivi a mano `expected.out` (con LF finale) ed `expected.code` (`0`), ragionando sulla specifica — **non** generarli alla cieca con `--update`, altrimenti fissi i bug del runtime;
4. copia `expected.out` in `expected.golden`;
5. `node conformance/run.mjs` deve stampare `PASS <nome>`;
6. se il caso passa anche sul seed Windows, aggiungilo a `$cases` nel job `seed-windows` di `.github/workflows/ci.yml`.

```
mkdir conformance/core/cases/somma_lista
cat > conformance/core/cases/somma_lista/program.td <<'TD'
xs = [3, 4, 5]
tot = 0
i = 1
while i <= length(xs)
    tot += xs[i]
    i += 1
end
println(tot)
TD
printf '12\n' > conformance/core/cases/somma_lista/expected.out
cp conformance/core/cases/somma_lista/expected.out conformance/core
printf '0' > conformance/core/cases/somma_lista/expected.code
node conformance/run.mjs | grep somma_lista          # PASS  somma_
```

I golden Full (`conformance/cases/`) sono **congelati**: non vanno rigenerati finché non esiste un'implementazione Full.

---

39. La CLI di sviluppo `triad-cli`

39.1 Uso

comando	cosa fa
`triad-cli status`	versione Node, piattaforma, seed trovato, root del progetto
`triad-cli test conformance [-c DIR] [-u]`	esegue `conformance/run.mjs` (default Core, `--skip liste`)
`triad-cli test run`	alias della conformance Core (i vecchi test Julia non esistono più)
`triad-cli ai verify [-t DIR] [-v]`	esegue le `solution.triad` di `ai/tasks/` sul seed nativo e le confronta con i golden
`triad-cli seed generate windows|linux [-o FILE]`	esegue il generatore del seed e stampa lo SHA-256 del dump
`triad-cli build standalone`	non disponibile: stampa un messaggio

Avvio: `node cli/triad-cli.mjs …` (dopo `cd cli && npm install`), i launcher `bin/triad-cli` / `bin\triad-cli.cmd` (installano le dipendenze al primo avvio e si fermano se `npm install` fallisce), oppure l'eseguibile standalone della release.

39.2 Struttura del codice

```
cli/
  triad-cli.mjs            definizione dei comandi con commander
  lib/utils.mjs            findProjectRoot, findSeed, runSeed, runN
  lib/commands/test.mjs    handleTestRun, handleTestConformance
  lib/commands/ai.mjs      handleAIVerify
  lib/commands/seed.mjs    handleSeedGenerate (windows/linux)
  lib/commands/build.mjs   handleBuildStandalone
  build-exe.mjs            build dell'eseguibile Node SEA (esbuild
```

- `findProjectRoot()` cerca la root (una cartella con `conformance/run.mjs` e `seed/`) partendo da `TRIAD_HOME`, dalla cartella del modulo, da quella dell'eseguibile e dalla cartella corrente.
- Ogni handler restituisce `true` / `false`; `triad-cli.mjs` lo traduce in exit code 0/1.

39.3 Aggiungere un comando

1. crea `cli/lib/commands/<nome>.mjs` con una funzione `handle<Nome>(options)` che restituisce un booleano e usa `projectPath`, `runNode`, `runSeed` da `utils.mjs`;
2. registralo in `triad-cli.mjs`:

```
import { handleLint } from './lib/commands/lint.mjs';

program
  .command('lint')
  .description('Controlla i file .triad')
  .option('-v, --verbose', 'Verbose output')
  .action(async (options) => {
    const success = await handleLint(options);
    process.exit(success ? 0 : 1);
  });
```

3. aggiungi la riga in `status` e documentalo in `cli/README.md`.

39.4 Eseguibile standalone

```
cd cli && npm install
npm run build:exe                          # dist/triad-cli per la p
node build-exe.mjs path/to/win/node.exe    # dist/triad-cli.exe per
```

Il binario Node passato determina la piattaforma; deve avere la stessa versione del Node che esegue la build.

---

40. Benchmark AI (`ai/`)

Misura quanto bene un LLM scrive Triad-Core.

- `ai/SYSTEM_PROMPT.md`: prompt di sistema con il sottoinsieme ammesso (prudente: vieta liste, `string()`, `elseif`, `return` perché il seed non li supportava tutti);
- `ai/tasks/NN_nome/`: `task.md` (richiesta), `expected.out`, `solution.triad` (soluzione di riferimento);
- verifica: `node cli/triad-cli.mjs ai verify` (o `--tasks ai/tasks/01_somma`): `PASS` / `FAIL`, `SKIP` se manca la soluzione;

- `ai/jev_audit.mjs`, `ai/jev_prompt.md`: audit separato (vedi `PROMPT_JEV.md`).

Aggiungere un task: crea `ai/tasks/06_nome/` con `task.md` (output atteso descritto con precisione), `expected.out` e una `solution.triad` di riferimento che passa. Esegui il codice generato da un modello solo in un ambiente isolato.

---

41. Editor, launcher e sito

- **VS Code** (`editors/vscode/`): `package.json` dichiara il linguaggio `triad` (estensioni `.triad`, `.td`); `syntaxes/triad.tmLanguage.json` è la grammatica TextMate (`source.triad`) con le regole `keywords`, stringhe con interpolazione, numeri, `:-`, shebang; `language-configuration.json` gestisce commenti e parentesi. Pacchetto: `cd editors/vscode && npx @vscode/vsce package`.
- **Vim/Neovim** (`editors/vim/`): `ftdetect/triad.vim` e `syntax/triad.vim`.
- **Linguist** (`editors/linguist/`): voce `languages.yml` pronta per una PR a GitHub Linguist.
- Se aggiungi una parola chiave allo standard, aggiornala in **tutte** le grammatiche.
- **Launcher** (`bin/`): `triad` (POSIX, risolve i symlink e calcola `TRIAD_HOME`) e `triad.cmd` (Windows, file CRLF); vedi §2.4.
- **Sito** (`docs/`): HTML statico pubblicato su GitHub Pages; `docs/assets/js/app.js` carica i Markdown del repository con `marked` (es. `docs/pages/tutorial.html` carica questo file). Le ancore dell'indice usano `<a id="…">` espliciti perché funzionino anche lì.

---

42. Integrazione continua

`.github/workflows/ci.yml`, a ogni push e pull request:

job	cosa verifica
`conformance-core` (Ubuntu, Node 20)	`node conformance/run.mjs --cases conformance/core/cases --skip liste` con l'interprete JS
`seed-windows`	decodifica `seed.hex`, esegue 33 casi Core sul PE confrontando stdout, probe `test_get.td` (atteso `20`), smoke test `selfhost/main.td` (atteso `Value: 10`), hash di `seed.hex`
`seed-linux`	decodifica l'ELF E0, lo esegue e ne verifica l'hash

`pages.yml` e `static.yml` pubblicano `docs/` su GitHub Pages a ogni push su `main`.

Il profilo Full **non è in CI** perché non è implementato: non presentarlo come verde.

---

43. Flusso di contributo

Checklist per tipo di modifica:

modifica	da fare
comportamento del linguaggio	aggiorna `SPEC.md` / `SPEC-CORE.md`, `CHANGELOG.md` e, se normativa, `VERSION`; aggiungi o aggiorna casi di conformance
interprete JS	resta fedele a SPEC-CORE; `node conformance/run.mjs` verde
seed Windows	rigenera `seed.hex`, aggiorna hash in CI e nei README, prova su Windows
S1 (`selfhost/`)	gira su `seed.exe`; se cambia l'output di `main.td`, aggiorna lo smoke test in CI
CLI	handler che restituisce un booleano; `cli/README.md`; ricostruisci l'eseguibile se serve
editor	grammatiche VS Code e Vim allineate alle parole chiave di SPEC §1
documentazione	questa wiki, `README.md`, `STATUS.md` coerenti con ciò che gira davvero

Regole generali: non dichiarare funzionante ciò che non è verificato; i golden si scrivono ragionando sulla specifica; i nuovi strumenti di sviluppo non devono entrare nella catena S0 → S4.

Parte V — Riferimento

---

44. Grammatica Triad-Core (EBNF)

Da `SPEC-CORE.md` §4:

```
program  = { sep } { stmt sep { sep } } ;
stmt     = funcdef | "return" [ expr ] | "break" | "continue" | exp
funcdef  = "function" NAME "(" [ NAME { "," NAME } ] ")" block "end
         | NAME "(" [ NAME { "," NAME } ] ")" "=" expr ;
expr     = assegnamento ;
assegnamento = NAME ( "=" | "+=" | "-=" | "*=" ) expr | ternario ;
ternario = or_expr [ "?" expr ":" expr ] ;
or_expr  = and_expr { "||" and_expr } ;
and_expr = cmp_expr { "&&" cmp_expr } ;
cmp_expr = sum_expr { ( "==" | "!=" | "<" | "<=" | ">" | ">=" ) sum
sum_expr = mul_expr { ( "+" | "-" ) mul_expr } ;
mul_expr = pow_expr { ( "*" | "%" ) pow_expr } ;
pow_expr = unary { "^" unary } ;                    (* destra *)
unary    = ( "-" | "!" ) unary | postfix ;
postfix  = primary { "(" args ")" | "[" expr "]" } ;
primary  = INT | STRING | SYMBOL | "true" | "false" | "nothing" | N
         | "(" expr ")"
         | "[" [ expr { "," expr } ] "]"
         | "(" NAME { "," NAME } ")" "->" expr
         | "if" expr block { "elseif" expr block } [ "else" block ]
         | "while" expr block "end"
         | "let" [ NAME "=" expr { "," NAME "=" expr } ] block "end
         | "begin" block "end"
         | "function" "(" [ NAME { "," NAME } ] ")" block "end" ;
block    = { stmt sep { sep } } ;                   (* fino a end/e
```

La grammatica Full (Pratt, 12 livelli) è in `SPEC.md` §2.

---

45. Schede rapide Core e Full

Triad-Core (eseguibile oggi):

```
assegnamento:  x = e    x += e    x -= e    x *= e    xs[i] = e
funzione:      f(a, b) = e
               function f(a, b) ... end        (anche !io, ignorato
anonima:       x -> e    (a, b) -> e    function (a) ... end
condizione:    if c ... elseif c ... else ... end    c ? a : b
ciclo:         while c ... end    break    continue
blocchi:       let a = 1, b = 2 ... end    begin ... end
uscita:        return e
liste:         [1, 2]    xs[i] (da 1)    length(xs)    push!(xs, v)
stringhe:      "a" * "b"    length(s)    string(x)
conversioni:   Int("42")    Symbol("s")    typeof(x)
I/O:           print(...)    println(...)    ARGS
bootstrap:     readfile(p)  strlen(s)  strbyte(s, i)  strsub(s, a,
```

Triad-Full (specifica):

```
struttura:   struct Nome; x::T; y::T; end
funzione:    function f(a::T)::R !eff ... end
fatto:       fact p(:a, :b)
regola:      rule p(?x, ?y) :- q(?x), r(?y)
query:       solve p(?x) limit N depth D steps S trace
destruttura: (a, b) = [1, 2]        for (k, v) in [[1,2],[3,4]] ...
ciclo for:   for v in it ... end    1:n    xs[2:end]    [e for x in
try/catch:   try b catch e h end / try b end
contratto:   requires c / ensures c
budget:      with budget(steps = N) ... end
traccia:     with trace ... end / solve p trace
effetto:     function f() !io ... end
macro:       macro m(a) ... end / @m(e)
quote:       :(espr) / quote ... end / $espr
eval:        eval(codice)
```

46. Debug e problemi comuni

Core (oggi):

sintomo	causa probabile	soluzione
`parse error: expected RPAREN`	`1.5`, `/`, `() -> …` o altra sintassi Full	resta nel sottoinsieme Core (§4)
`"x = $x"` stampato letteralmente	l'interpolazione è solo Full	`"x = " * string(x)`
`error: if cond`	condizione non `Bool`	usa un confronto (`x != 0`)
`error: < type error` su `a < b < c`	confronti concatenati	`a < b && b < c`
una funzione non aggiorna un globale	le funzioni nominate assegnano in locale	restituisci il valore o usa una lista (§16.4)
`error: undefined variable f`	funzione chiamata prima della definizione	definiscila più in alto
`FAIL … (golden mancante …)`	checkout senza `expected.out`	`git pull` ; controlla `.gitignore`
il seed esce con 1 senza messaggi	funzionalità non supportata dal seed	prova con l'interprete JS; vedi §15
il seed Windows viene bloccato	falso positivo dell'antivirus	escludi `seed\windows-x86_64`

Full (specifica):

sintomo	causa probabile	soluzione
`Err(:syntax)` "atteso fine istruzione"	manca `end` o un separatore	aggiungi `end` o `;`
`Err(:undef)`	nome non definito	definiscilo, o usa `:nome` per un simbolo
`Err(:method)` / `Err(:ambiguous)`	nessun metodo / più metodi ugualmente specifici	aggiungi o ridisegna le firme
`Err(:bounds)` in destrutturazione	arità diversa dal pattern	liste di lunghezza esatta
`Err(:effect)`	`!io` / `!state` chiamata da funzione pura	dichiara l'effetto nel chiamante
`Err(:budget)` in `solve`	troppi passi	aumenta `steps` o usa `limit` / `depth`

Strumenti Full previsti: `triad --tree file`, `sexpr(codice)`, `parse` / `unparse`, `with trace … end`, `solve … trace`, `try … end` con `.kind` / `.msg` / `.data`.

---

47. Glossario

- **S0 / seed**: interprete Triad-Core in codice macchina (`seed/`).
- **S1**: interprete di tutto Triad scritto in Triad-Core (`selfhost/`).
- **S2 / S3 / S4**: compilatore nativo, self-hosting, runtime completo (vedi `BOOTSTRAP.md`).
- **E0 / E1 / …**: stadi del port Linux del seed (milestone M7).
- **Triad-Core / Triad-Full**: i due profili di conformità.
- **Interprete JS**: `seed/dev/triad_core.cjs` , riferimento Core per sviluppo e CI.
- **Golden**: output atteso committato (`expected.out` , `expected.code`).
- **Primitivi di bootstrap**: `readfile` , `strlen` , `strbyte` , `strsub` , `exit` (solo Core).
- **Lista taggata**: record Core del tipo `[:Tag, campo1, …]` .

- **Node**: l'unico tipo di albero di Triad Full (`head` , `args`): codice e dati coincidono.
- **Solution**: `{nome = valore, …}` restituita da `solve` ; `s.nome` o `s[:nome]` .
- **Err**: errore come dato (`kind` , `msg` , `data`).
- **Traced**: `(result, trace)` restituito da `with trace` / `solve … trace` .
- **Occurs-check**: l'unificazione rifiuta `?x = f(?x)` .
- **Multiple dispatch**: scelta del metodo più specifico tra gli applicabili.
- **Effetti dichiarati**: `!io` , `!state` nella firma; controllo dinamico.
- **Contratti**: `requires` (pre) / `ensures` (post, con `result`).
- **Budget**: `with budget(steps = N)` ; l'esaurimento è `Err(:budget)` .
- **Destrutturazione**: `(a, b) = lista` , `for (k, v) in lista_di_liste` .

---

48. Stato, limiti noti e fix storici

Stato (dettagli in `STATUS.md`):

- Core: 37/37 casi con l'interprete JS; 33 casi sul seed Windows in CI.
- Full: 12 casi golden congelati, passa solo `errore` .
- S1: prototipi in `selfhost/` , smoke test in CI; non ancora conformance.
- S2–S4: non iniziati. Seed Linux: E0.
- CLI `triad` : solo esecuzione di file (REPL e opzioni di SPEC §14 da fare).

Limiti dello standard (SPEC §15): argomenti con nome (salvo `budget(steps = …)`), struct mutabili, tipi astratti utente e parametrici, macro senza parentesi, quote annidate a più livelli, cut di Prolog non sono previsti; il controllo degli effetti è dinamico.

Fix storici applicati alla vecchia implementazione Julia; il comportamento "dopo" è quello richiesto da SPEC.md per ogni implementazione:

#	fix	dopo
1	round-trip del range	`unparse(1:n)` produce `"1:n"` (senza spazi)
2	`unparse` di `:destructure` / `:dfor`	hanno rami propri e fanno round-trip
3	`xs[end] = v`	`end` è legato anche in scrittura: `xs[end] = 7` funziona
4	escape sconosciuti	solo `\n \t \" \\ \$`; altri → `Err(:syntax)` con riga e colonna
5	commenti `#=` annidati	la profondità è contata; `#=` non chiuso → errore `incomplete`
6	lambda con parametri tipati	`function (x::Int) x end` controlla i tipi: `f(2.0)` → `Err(:type)`

---

49. FAQ

Serve Julia? No. L'implementazione Julia è stata rimossa; Triad gira sul seed in codice macchina (Windows) o sull'interprete JS di sviluppo.

Perché i miei esempi con `struct` o `fact` non girano? Sono Triad-Full: il runtime attuale esegue solo Triad-Core. Vedi la tabella in testa alla wiki.

`.td` o `.triad`? `.triad` è canonica; `.td` è accettata come alias (ed è anche l'estensione di LLVM TableGen, quindi per il riconoscimento esterno conviene `.triad`).

Come verifico che un programma sia Core "puro"? Eseguilo con l'interprete JS e controlla di non usare primitivi di bootstrap se deve valere anche per Full; per i casi normativi aggiungilo alla conformance Core.

Perché `selfhost/*.td` falliscono con l'interprete JS? Usano `get(xs, i)`, helper non standard del seed Windows; vanno eseguiti con `seed.exe`.

Dove propongo una modifica al linguaggio? Nello standard (`SPEC.md` / `SPEC-CORE.md`) con voce in `CHANGELOG.md` e casi di conformance; poi nelle

implementazioni.

Wiki mantenuta insieme allo standard: quando cambia `SPEC.md`, `SPEC-CORE.md`, il runtime o gli strumenti, aggiorna il capitolo corrispondente. Gli esempi Core marcati **Output:** si verificano eseguendoli con `node seed/dev/triad_core.cjs`.

Il libro di Triad

Triad 2.0 Ã¨ un linguaggio di programmazione autonomo, definito da una specifica normativa indipendente da qualsiasi implementazione. Non Ã¨ un dialetto di Julia, Prolog o Lisp, pur avendo preso da ciascuno di essi idee fondamentali: blocchi `function â€¦ end` e tipi con `::` da Julia, unificazione con occurs-check, backtracking e `solve` con variabili logiche `?x` da Prolog, codice come dato e macro da Lisp. CiÃ² che Ã¨ proprio di Triad, e non in prestito, Ã¨ l'integrazione dei tre modelli â€” imperativo, logico, metaprogrammatorio â€” in una sola sintassi a blocchi, accompagnata da errori come dati (`Err`), contratti (`requires` / `ensures`), budget (`with budget`), tracce (`with trace`) ed effetti dichiarati (`!io`, `!state`) come parte del linguaggio e non di una libreria.

Il linguaggio si presenta in due profili di conformitÃ . Triad-Core Ã¨ il sottoinsieme di bootstrap: valori, funzioni, chiusure, controllo di flusso, liste, stringhe, I/O. Triad-Full Ã¨ lo standard completo: tipi dichiarati, multiple dispatch, logica integrata, macro, contratti, budget, tracce, effetti. Ogni programma Core Ã¨ anche un programma Full valido e deve comportarsi allo stesso modo. Il runtime di riferimento Ã¨ il seed S0: un interprete Triad-Core scritto direttamente in codice macchina x86-64 (PE su Windows, ELF su Linux), senza alcun linguaggio ospite. Il piano di bootstrap prevede S1 (un interprete Triad scritto in Triad-Core), S2 (compilatore nativo), S3 (self-hosting) e S4 (paritÃ cross-platform).

Questo libro Ã¨ il percorso di studio del linguaggio per come Ã¨ definito dallo standard. La prima parte, Fondamenti, introduce il lessico, i valori e i tipi, il controllo di flusso, le funzioni, il dispatch, la logica, il codice come dato, gli errori, i contratti, il budget, le tracce e gli effetti. La seconda, Triad-Core, copre il sottoinsieme eseguibile oggi dall'interprete di riferimento, con tutti i 37 casi di conformance. La terza, Triad-Full, presenta i costrutti dello standard completo, oggi studiabili ma non ancora eseguibili dal runtime. La quarta,

Dentro il progetto, accompagna il lettore nel codice del repository: il seed, la suite di conformance, la CLI, gli strumenti per gli editor.

Il progetto Ã¨ rilasciato sotto licenza MIT. La specifica normativa Ã¨ in SPEC.md (standard 0.2.0), il sottoinsieme di bootstrap in SPEC-CORE.md, la suite di conformance in conformance/ con golden verificabili. Il repository include anche un benchmark per la generazione di codice Triad da LLM (ai/), grammatiche per VS Code e Vim, e un sito con la curva di apprendimento. L'obiettivo Ã¨ un linguaggio autonomo e riconosciuto: standard proprio, conformance eseguibile, estensione .triad, tooling editor, bootstrap senza linguaggi ospiti e, infine, self-hosting.

La suite di conformance Ã¨ il contratto eseguibile del linguaggio: 37 casi Triad-Core e 12 casi Triad-Full, ognuno con golden verificabili byte per byte, inclusi l'exit code. Il runner Ã¨ scritto in Node e non dipende da Julia: il seed Windows passa 33 casi Core sul PE, il seed Linux Ã¨ in fase E0. Ogni caso dichiara il profilo (Core o Full) e un golden mancante fa fallire il caso, cosÃ¬ che nessuna implementazione possa registrare comportamenti scorretti come attesi. I test del progetto (unit e end-to-end) coprono parser, valutatore, unificazione, macro e CLI.

Gli strumenti per gli editor includono grammatiche per VS Code e Vim/Neovim e una voce per GitHub Linguist, cosÃ¬ che i file `.triad` (canonica) e `.td` (alias storico) siano riconosciuti ovunque. Il sito del progetto espone la curva di apprendimento, i grafici delle fasi e una demo animata della battitura di un programma Triad-Core eseguito dall'interprete. Il repository Ã¨ organizzato per essere esplorato: esempi progressivi in `examples/`, task guidati in `ai/tasks/`, benchmark LLM in `ai/`. Ogni modifica Ã¨ accompagnata da test e da aggiornamenti a STATUS.md, BOOTSTRAP.md e CHANGELOG.md.

Questo libro Ã¨ scritto per lo standard 0.2.0. In caso di divergenza tra manuale e specifica, prevale la specifica: SPEC.md e SPEC-CORE.md sono la fonte di veritÃ normativa, e questo volume ne Ã¨ il percorso di studio, con spiegazioni, esempi ed esercizi.